

Computing IV Sec 201: Project Portfolio

Jayden Danquah

Spring 2025

Contents

1	PS0: Hello World with SFML	2
2	PS1: Linear Feedback Shift Register and Image Encoding	5
3	PS2: Recursive Graphics (Triangle Fractal)	15
4	PS3: N-Body Simulation	19
5	PS4: Sokoban	31
6	PS5: DNA String Alignment	45
7	PS6: RandWriter	53
8	PS7: Kronos InTouch Log Parsing	60

Time to Complete Portfolio: 5 days

1 PS0: Hello World with SFML

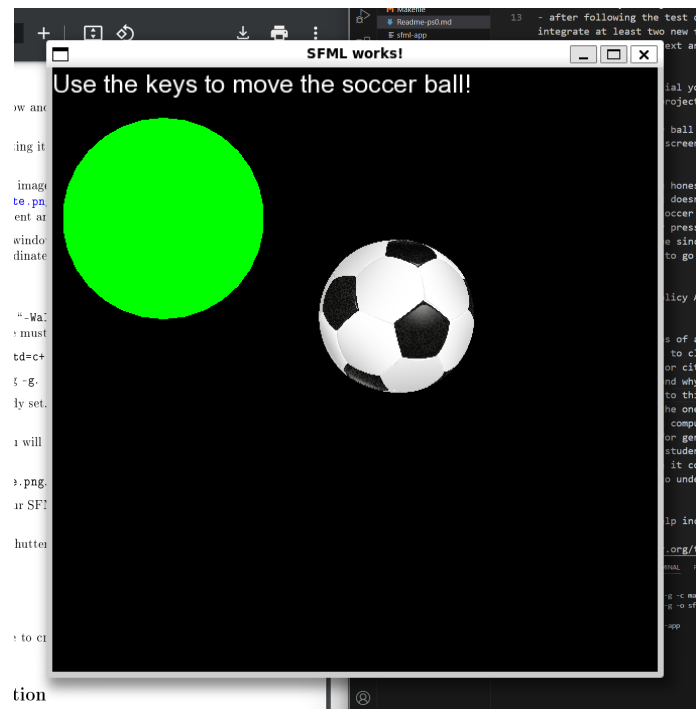


Figure 1: Window produced from running the tutorial code

1.1 Assignment Overview

Before we were given our first project, we had to set up our build environment by installing Ubuntu and other tools required to complete future assignments, such as cpplint, valgrind, gdb, and the SFML and Boost libraries. Once that was finished, our objective was to copy the tutorial code from the SFML website and add a sprite that responds to keyboard commands, as well as another feature of our choosing.

1.2 My Coding Approach

The tutorial code provided to us drew a simple green circle in the middle of a window screen, with the header reading “SFML works!” To expand the code, I added a soccer ball and coded keyboard arguments to have it move horizontally and vertically along the window screen, which I also enlarged from its original dimensions. For my additional feature, I created a simple message instructing the user to move the soccer ball sprite using the arrow keys.

1.3 What I Learned and Other Reflections

With this being my introduction to using the SFML library, I did not want to go overboard when adding extra features to my code, but I also wanted to give myself a "taste" of what functions, members, and variables I could expect to use in future assignments. Understanding how to load a .PNG file into a sprite and modify it was my main priority, as well as how the `window.isOpen()` loop works.

1.4 Source Code

Listing 1: Makefile

```
1 CC = g++
2 CFLAGS = --std=c++20 -Wall -Werror -pedantic -g
3 LIB = -lsfml-graphics -lsfml-audio -lsfml-window -lsfml-system -
      lboost_unit_test_framework
4 # Your .hpp files
5 DEPS =
6 # Your compiled .o files
7 OBJECTS =
```

```

8 # The name of your program
9 PROGRAM = sfml-app
10
11 .PHONY: all clean lint
12
13 all: $(PROGRAM)
14
15 # Wildcard recipe to make .o files from corresponding .cpp file
16 %.o: %.cpp $(DEPS)
17     $(CC) $(CFLAGS) -c $<
18
19 $(PROGRAM): main.o $(OBJECTS)
20     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
21
22 clean:
23     rm *.o $(PROGRAM)
24
25 lint:
26     cpplint *.cpp *.hpp

```

Listing 2: main.cpp

```

1 // Copyright 2025 by Jayden Danquah
2
3 #include <SFML/Graphics.hpp>
4
5 int main(int argc, char* argv[]) {
6     sf::RenderWindow window(sf::VideoMode(600, 600), "SFML works!");
7
8     // creates text to display
9     sf::Font font;
10    if (!font.loadFromFile("Arial.ttf"))
11        return EXIT_FAILURE;
12    sf::Text text("Use the keys to move the soccer ball!", font, 25);
13
14    // creates circle and colors it green
15    sf::CircleShape shape(100.f);
16    shape.setFillColor(sf::Color::Green);
17    // creates gap between text and circle
18    shape.setPosition({10.f, 50.f});
19
20    // loads sprite.png
21    sf::Texture texture;
22    if (!texture.loadFromFile("sprite.png"))
23        return EXIT_FAILURE;
24    // creates soccer ball sprite
25    sf::Sprite sprite(texture);
26    // scales image
27    sprite.setScale({0.0625f, 0.0625f});
28    sprite.setPosition({20.f, 100.f});
29
30    while (window.isOpen()) {
31        sf::Event event;
32        while (window.pollEvent(event)) {
33            if (event.type == sf::Event::Closed) {
34                // close window
35                window.close();
36            }
37        }
38        window.clear();

```

```
39     window.draw(text);
40     window.draw(shape);
41     window.draw(sprite);
42     window.display();
43
44     // code to make soccer ball move
45     if (sf::Keyboard::isKeyPressed(sf::Keyboard::Left)) {
46         // move left
47         sprite.move({-0.25f, 0.f});
48     } else if (sf::Keyboard::isKeyPressed(sf::Keyboard::Right)) {
49         // move right
50         sprite.move({0.25f, 0.f});
51     } else if (sf::Keyboard::isKeyPressed(sf::Keyboard::Up)) {
52         // move up
53         sprite.move({0.f, -0.25f});
54     } else if (sf::Keyboard::isKeyPressed(sf::Keyboard::Down)) {
55         // move down
56         sprite.move({0.f, 0.25f});
57     }
58 }
59
60 return 0;
61 }
```

2 PS1: Linear Feedback Shift Register and Image Encoding

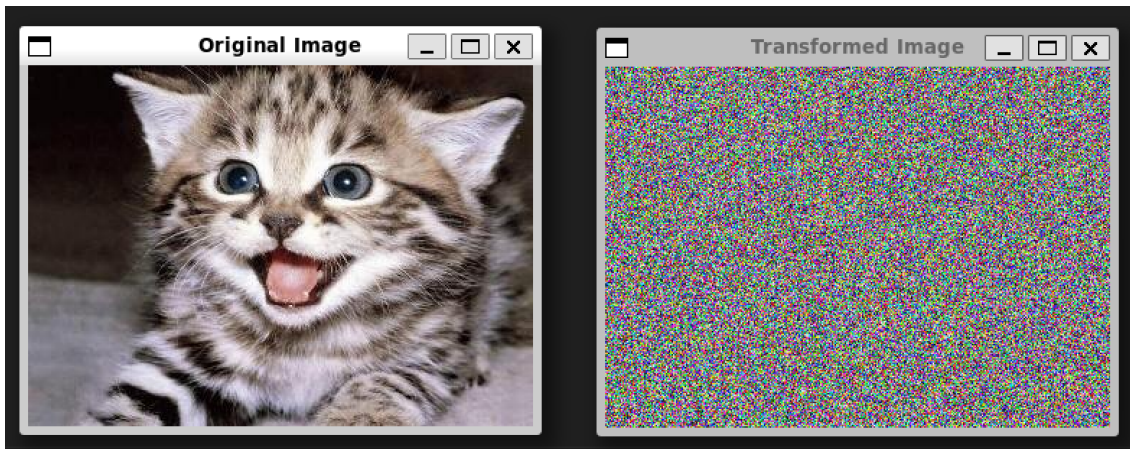


Figure 2: Encrypting an image file

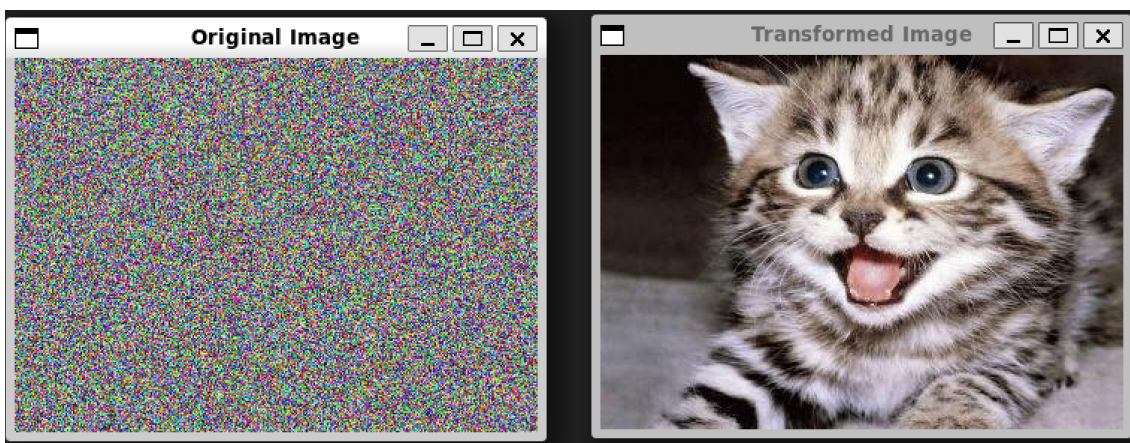


Figure 3: Decrypting an image file

2.1 Assignment Overview

Our first assignment consisted of two parts. In the first part, our objective was to write a program that produces pseudo-random bits by simulating a linear feedback shift register (LFSR). In the second part, the code we wrote was then used to implement a simple form of encryption for digital pictures. Both parts also required us to write test.cpp files using the Boost library framework before writing our primary code; these files were submitted first and ran through an autograder using our instructor's implementation.

2.2 My Coding Approach

I ultimately redid the first part of this assignment (one of three assignments in this portfolio that were redone) due to several errors and misinterpretations in my code. I originally represented the LFSR as a vector of integers holding the seed (the sequence used to initialize the register). A major misstep came in the implementation of my `step()` function, which neglected to XOR the most significant bit with the designated tap positions, and instead XOR'd the first and last bits to generate either a 0 or 1. This error also affected the output of my `generate()` function, with the numbers returned after running `step()` unequal to the sample output shown in the assignment instructions. In my redo of the first part, I switched from a vector-based representation of the LFSR to a deque-based one, so that the `push_back()` and `erase()` functions could properly shift the bits and so I could designate the proper tap positions.

The second part of the program, though built from my flawed implementation, used the LFSR class I had written in creating a new class called `PhotoMagic`. The primary function in this new class, `transform()`, uses the `generate()` function to get new red, blue, and green

(RGB) color values, which then overwrite each pixel of an image sent into the program. The user passes in their image, the destination for the encrypted image, and an LFSR seed, and the program returns two windows showing the original and encrypted images; by reversing the order of files while using the same seed, the image can be decrypted as well.

2.3 What I Learned and Other Reflections

While I was familiar with unit testing and scripting from other programming languages I have worked with, I personally prefer to develop my tests as I worked through my actual code. The starter.hpp and test.cpp files provided in this assignment were helpful in shaping the general structure of my implementation, but when it came to actually writing the test.cpp, I often used naming conventions that deviated from what was established in the starter file or invoked class functions and variables that did not exist in the autograder implementation, which caused me to lose many points. This was an issue I continued to struggle with throughout the next couple of assignments, but I soon got in the habit of writing the bulk of my tests before diving headfirst into the implementation.

2.4 Source Code

Listing 3: Makefile

```
1 CC = g++
2 CFLAGS = --std=c++17 -Wall -Werror -pedantic -g
3 LIB = -lsfml-graphics -lsfml-audio -lsfml-window -lsfml-system -
    lboost_unit_test_framework
4
5 # .hpp files
6 DEPS =
7 # Compiled .o files
8 OBJECTS = FibLFSR.o PhotoMagic.o
9 # Name of program
10 PROGRAM = FibLFSR
11
12 .PHONY: all clean lint
13
14 all: test PhotoMagic PhotoMagic.a
15
16 %.o: %.cpp $(DEPS)
17     $(CC) $(CFLAGS) -c $<
18
19 $(PROGRAM): main.o $(OBJECTS)
20     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
21
22 test: test.o $(OBJECTS)
23     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
24
25 PhotoMagic: main.o $(OBJECTS)
26     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
27
28 PhotoMagic.a: $(OBJECTS)
29     ar rcs $@ $^
30
31 clean:
32     rm -f *.o test PhotoMagic.a
33
34 lint:
35     cpplint *.cpp *.hpp
```

Listing 4: main.cpp

```

1 // Copyright 2025
2 // By Jayden Danquah
3
4 // g++ -o pixels pixels.cpp -lsfml-graphics -lsfml-window
5
6 #include <iostream>
7 #include <string>
8 #include <SFML/System.hpp>
9 #include <SFML/Window.hpp>
10 #include <SFML/Graphics.hpp>
11 #include "PhotoMagic.hpp"
12 #include "FibLFSR.hpp"
13
14 int main(int argc, char* argv[]) {
15     if (argc != 4) {
16         std::cerr << "Usage: " << argv[0] <<
17             " input-file.png output-file.png password" << std::endl;
18         return -1;
19     }
20
21     // input file
22     std::string inputFile = argv[1];
23
24     // output file
25     std::string outputFile = argv[2];
26
27     // binary password
28     std::string password = argv[3];
29
30     // create LFSR
31     PhotoMagic::FibLFSR lfsr(password);
32
33     // load image
34     sf::Image image;
35     if (!image.loadFromFile(inputFile)) {
36         std::cerr << "ERROR - failed to load image" << std::endl;
37         return -1;
38     }
39
40     // display original and transformed images
41     PhotoMagic::display(image, &lfsr);
42     // save transformed image
43     if (!image.saveToFile(outputFile)) {
44         std::cerr << "ERROR - failed to save image" << std::endl;
45         return -1;
46     }
47
48     return 0;
49 }

```

Listing 5: FibLFSR.cpp

```

1 // Copyright 2025
2 // By Jayden Danquah
3
4 #include "FibLFSR.hpp"
5 #include <deque>
6 #include <string>
7 #include <stdexcept>
8

```

```

9 // constructor to create LFSR with given initial seed
10 PhotoMagic::FibLFSR::FibLFSR(std::string seed) {
11     if (seed.empty()) {
12         throw std::invalid_argument("ERROR - empty seed");
13     }
14     if (seed.length() != 16) {
15         throw std::invalid_argument("ERROR - seed length must be less"
16             "than 16");
17     }
18     for (char c : seed) {
19         if (c != '0' && c != '1') {
20             throw std::invalid_argument("ERROR - invalid seed");
21         }
22         raw_seed.push_back(c - '0');
23     }
24 }
25
26 // simulate one step and return new bit as 0 or 1
27 int PhotoMagic::FibLFSR::step() {
28     // compute new bit using tap positions 13, 12, and 10
29     // using LSB logic: 15 -> 0 (MSB), 13 -> 2, 12 -> 3, 10 -> 5
30     int leftmost = raw_seed[0];
31     int new_bit = leftmost ^ raw_seed[2] ^ raw_seed[3] ^ raw_seed[5];
32
33     // insert new bit at end
34     raw_seed.push_back(new_bit);
35
36     // shift left by removing first bit
37     raw_seed.pop_front();
38
39     return new_bit;
40 }
41
42 // simulate k steps and return k-bit integer
43 int PhotoMagic::FibLFSR::generate(int k) {
44     if (k <= 0) {
45         throw std::invalid_argument("ERROR - k must be positive integer");
46     }
47     int result = 0;
48     for (int i = 0; i < k; i++) {
49         // shift result left and append new bit
50         result <<= 1;
51         result ^= step();
52     }
53     return result;
54 }
55
56 std::ostream& PhotoMagic::operator<<(std::ostream& out,
57     const PhotoMagic::FibLFSR& lfsr) {
58     for (int bit : lfsr.seed()) {
59         out << bit;
60     }
61     return out;
62 }

```

Listing 6: FibLFSR.hpp

```

1 // Copyright 2025
2 // By Jayden Danquah
3

```

```

4  #pragma once
5
6  #include <iostream>
7  #include <deque>
8  #include <string>
9
10 namespace PhotoMagic {
11 class FibLFSR {
12 public:
13     // constructor to create LFSR with given initial seed
14     explicit FibLFSR(std::string seed);
15
16     // raw_seed accessor
17     const std::deque<int>& seed() const { return raw_seed; }
18
19     // retrives state
20     int state() const { return raw_seed.back(); }
21
22     // simulate one step and return new bit as 0 or 1
23     int step();
24
25     // simulate k steps and return k-bit integer
26     int generate(int k);
27 private:
28     // holder for raw seed
29     std::deque<int> raw_seed;
30 };
31 std::ostream& operator<<(std::ostream& out, const FibLFSR& lfsr);
32 } // end namespace PhotoMagic

```

Listing 7: PhotoMagic.cpp

```

1  // Copyright 2025
2  // By Jayden Danquah
3
4  #include <algorithm>
5  #include <SFML/Graphics.hpp>
6  #include "PhotoMagic.hpp"
7  #include "FibLFSR.hpp"
8
9  void PhotoMagic::transform(sf::Image& img, FibLFSR* lfsr) {
10     // get image dimensions
11     unsigned int width = img.getSize().x;
12     unsigned int height = img.getSize().y;
13
14     // iterate through each pixel
15     for (unsigned int i = 0; i < width; i++) {
16         for (unsigned int j = 0; j < height; j++) {
17             // get pixel color
18             sf::Color color = img.getPixel(i, j);
19
20             // get R, G, B values
21             int r = color.r;
22             int g = color.g;
23             int b = color.b;
24
25             // get new R, G, B values
26             r = r ^ lfsr->generate(8);
27             g = g ^ lfsr->generate(8);
28             b = b ^ lfsr->generate(8);

```

```

29
30         // set new pixel color
31         img.setPixel(i, j, sf::Color(r, g, b));
32     }
33 }
34
35 // save image
36 img.saveToFile("output-file.png");
37 }
38
39 void PhotoMagic::display(sf::Image& img, FibLFSR* lfsr) {
40     // create windows
41     sf::RenderWindow window1(sf::VideoMode(img.getSize().x,
42     img.getSize().y), "Original Image");
43     sf::RenderWindow window2(sf::VideoMode(img.getSize().x,
44     img.getSize().y), "Transformed Image");
45
46     // create sprite of original image
47     sf::Texture texture1;
48     texture1.loadFromImage(img);
49     sf::Sprite sprite1(texture1);
50
51     // create a copy of the original image
52     sf::Image transformedImg = img;
53
54     // transform the copied image
55     transform(transformedImg, lfsr);
56
57     // create sprite of transformed image
58     sf::Texture texture2;
59     texture2.loadFromImage(transformedImg);
60     sf::Sprite sprite2(texture2);
61
62     // display images in separate windows
63     while (window1.isOpen() && window2.isOpen()) {
64         sf::Event event;
65         while (window1.pollEvent(event)) {
66             if (event.type == sf::Event::Closed) {
67                 window1.close();
68             }
69         }
70         while (window2.pollEvent(event)) {
71             if (event.type == sf::Event::Closed) {
72                 window2.close();
73             }
74         }
75         window1.clear();
76         window1.draw(sprite1);
77         window1.display();
78         window2.clear();
79         window2.draw(sprite2);
80         window2.display();
81     }
82
83     // delete lfsr
84     delete lfsr;
85
86     // save image
87     img.saveToFile("output-file.png");

```

```
88 }
```

Listing 8: PhotoMagic.hpp

```
1 // Copyright 2025
2 // By Jayden Danquah
3
4 #pragma once
5
6 #include <algorithm>
7 #include <SFML/Graphics.hpp>
8 #include "FibLFSR.hpp"
9
10 namespace PhotoMagic {
11 // transforms image using FibLFSR
12 void transform(sf::Image& img, FibLFSR* lfsr);
13
14 // display encrypted copy of image using LFSR
15 void display(sf::Image& img, FibLFSR* lfsr);
16 }
```

Listing 9: pixels.cpp

```
1 // Copyright 2025
2 // By Jayden Danquah
3
4 // pixels.cpp:
5 // using SFML to load a file, manipulate its pixels, write it to disk
6 // g++ -o pixels pixels.cpp -lsfml-graphics -lsfml-window
7
8 #include <SFML/System.hpp>
9 #include <SFML/Window.hpp>
10 #include <SFML/Graphics.hpp>
11
12 int main(int argc, char* argv[]) {
13     sf::Image image;
14     if (!image.loadFromFile("cat.jpg")) {
15         return -1;
16     }
17
18     // p is a pixel image.getPixel(x, y);
19     sf::Color p;
20
21     // create photographic negative of upper-left 200px square
22     for (int x = 0; x < 200; x++) {
23         for (int y = 0; y < 200; y++) {
24             p = image.getPixel(x, y);
25             p.r = 255 - p.r;
26             p.g = 255 - p.g;
27             p.b = 255 - p.b;
28             image.setPixel(x, y, p);
29         }
30     }
31     sf::Vector2u size = image.getSize();
32     sf::RenderWindow window(sf::VideoMode(size.x, size.y), "Meow");
33     sf::Texture texture;
34     texture.loadFromImage(image);
35     sf::Sprite sprite;
36     sprite.setTexture(texture);
37     while (window.isOpen()) {
38         sf::Event event;
```

```

39     while (window.pollEvent(event)) {
40         if (event.type == sf::Event::Closed) {
41             window.close();
42         }
43     }
44     window.clear(sf::Color::White);
45     window.draw(sprite);
46     window.display();
47 }
48
49 // fredm: saving a PNG segfaults for me, though it does properly
50 // write the file
51 if (!image.saveToFile("cat-out.bmp")) {
52     return -1;
53 }
54
55 return 0;
56 }

```

Listing 10: test.cpp

```

1  // Copyright 2025
2  // By Jayden Danquah
3
4  #include <iostream>
5  #include <string>
6  #include <vector>
7  #include "FibLFSR.hpp"
8  #include "PhotoMagic.hpp"
9  #define BOOST_TEST_DYN_LINK
10 #define BOOST_TEST_MODULE Main
11 #include <boost/test/unit_test.hpp>
12
13 using PhotoMagic::FibLFSR;
14
15 // test the step function
16 BOOST_AUTO_TEST_CASE(testStep) {
17     FibLFSR lfsr("1011011000110110");
18     BOOST_REQUIRE_EQUAL(lfsr.step(), 0);
19     BOOST_REQUIRE_EQUAL(lfsr.step(), 0);
20     BOOST_REQUIRE_EQUAL(lfsr.step(), 0);
21     BOOST_REQUIRE_EQUAL(lfsr.step(), 1);
22     BOOST_REQUIRE_EQUAL(lfsr.step(), 1);
23     BOOST_REQUIRE_EQUAL(lfsr.step(), 0);
24     BOOST_REQUIRE_EQUAL(lfsr.step(), 0);
25     BOOST_REQUIRE_EQUAL(lfsr.step(), 1);
26 }
27
28 // test the generate function
29 BOOST_AUTO_TEST_CASE(testGenerate) {
30     FibLFSR lfsr("1011011000110110");
31     std::vector<int> expected = {3, 6, 14, 24, 1, 13, 28};
32     for (size_t i = 0; i < expected.size(); i++) {
33         BOOST_REQUIRE_EQUAL(lfsr.generate(5), expected[i]);
34     }
35 }
36
37 // test the overloaded output stream operator
38 BOOST_AUTO_TEST_CASE(testOstream) {
39     FibLFSR lfsr("1011011000110110");

```

```

40     std::ostringstream output;
41     output << lfsr;
42     BOOST_TEST(output.str() == "1011011000110110");
43 }
44
45 // test constructor with empty seed
46 BOOST_AUTO_TEST_CASE(testEmptySeed) {
47     BOOST_CHECK_THROW(FibLFSR(""), std::invalid_argument);
48 }
49
50 // test constructor with invalid seed
51 BOOST_AUTO_TEST_CASE(testInvalidSeed) {
52     BOOST_CHECK_THROW(FibLFSR("10110110001101102"), std::invalid_argument);
53 }
54
55
56 // test image is properly encrypted and decrypted
57 BOOST_AUTO_TEST_CASE(testEncryption) {
58     sf::Image image;
59     if (!image.loadFromFile("test.jpg")) {
60         BOOST_FAIL("Failed to load test.jpg");
61     }
62     FibLFSR lfsr("1011011000110110");
63     PhotoMagic::transform(image, &lfsr);
64     image.saveToFile("test-enc.png");
65
66     sf::Image encryptedImage;
67     if (!encryptedImage.loadFromFile("test-enc.png")) {
68         BOOST_FAIL("Failed to load test-enc.png");
69     }
70     PhotoMagic::transform(encryptedImage, &lfsr);
71     encryptedImage.saveToFile("test-dec.jpg");
72
73     sf::Image originalImage;
74     if (!originalImage.loadFromFile("test.jpg")) {
75         BOOST_FAIL("Failed to load test.jpg");
76     }
77     sf::Image decryptedImage;
78     if (!decryptedImage.loadFromFile("test-dec.jpg")) {
79         BOOST_FAIL("Failed to load test-dec.jpg");
80     }
81     // compare pixel data of original and decrypted images
82     BOOST_REQUIRE_EQUAL(originalImage.getSize().x,
83         decryptedImage.getSize().x);
84     BOOST_REQUIRE_EQUAL(originalImage.getSize().y,
85         decryptedImage.getSize().y);
86
87     for (unsigned int i = 0; i < originalImage.getSize().x; ++i) {
88         for (unsigned int j = 0; j < originalImage.getSize().y; ++j) {
89             BOOST_CHECK(originalImage.getPixel(i, j) ==
90                 decryptedImage.getPixel(i, j));
91         }
92     }
93 }
94
95 // test input file is not found
96 BOOST_AUTO_TEST_CASE(testFileNotFound) {
97     sf::Image image;
98     BOOST_CHECK_THROW(image.loadFromFile("test1.jpg"), std::runtime_error);

```

```

99 }
100
101 // test output file is not found
102 BOOST_AUTO_TEST_CASE(testOutputFileNotFound) {
103     sf::Image image;
104     if (!image.loadFromFile("test.jpg")) {
105         BOOST_FAIL("Failed to load test.jpg");
106     }
107     FibLFSR lfsr("1011011000110110");
108     PhotoMagic::transform(image, &lfsr);
109     BOOST_CHECK_THROW(image.saveToFile("nonexistent_directory/test-enc.png"),
110         std::runtime_error);
111 }
112
113 // test display function
114 BOOST_AUTO_TEST_CASE(testDisplay) {
115     sf::Image image;
116     if (!image.loadFromFile("test.jpg")) {
117         BOOST_FAIL("Failed to load test.jpg");
118     }
119     FibLFSR lfsr("1011011000110110");
120     PhotoMagic::display(image, &lfsr);
121 }

```

3 PS2: Recursive Graphics (Triangle Fractal)

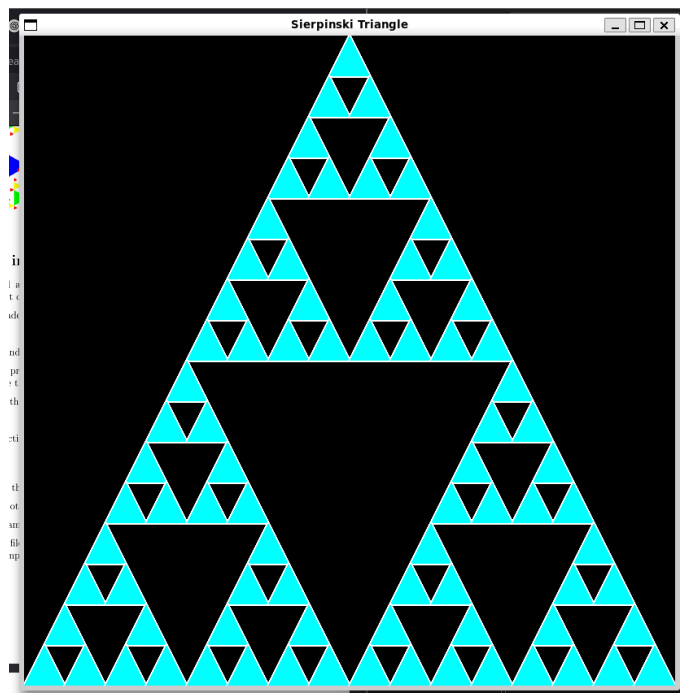


Figure 4: Window produced from running the code

3.1 Assignment Overview

In this assignment, our objective was to draw a fractal resembling the Sierpinski triangle (an equilateral triangle consisting of smaller "child" triangles, which in turn consist of even smaller triangles) using a recursive function. This was also the first assignment where we were allowed to work with a partner by way of pair programming, in which roles alternate between actively developing the code and revising it.

3.2 My Coding Approach

For my Triangle class, there were three primary functions I created. The constructor takes in the side length of the parent triangle as well as the depth of the recursion. The `base()` function draws the initial triangle and then calls upon a helper function, `fractal()`, to recursively draw child triangles derived from the parent triangle. In both functions, `sf::ConvexShape` is used to draw a triangle with three points representing the top, left, and right vertices. Within `fractal()`, the three child triangles are positioned within the larger parent triangle by calculating the midpoints of their left, right, and bottom sides; these values are then passed into `fractal()` itself, along with a recursion depth that was one unit smaller than what was originally passed into the program. For extra credit, I also set the colors of the triangle to teal with white outlines.

3.3 What I Learned and Other Reflections

Completing this assignment was difficult. My initial pass at the `fractal()` function attempted to draw the three child triangles in the event that the recursion depth was less than 0, but the resulting calls were not properly dividing the triangles, nor were they drawn correctly. For example, two points of the triangle would have proper connections to the parent triangle and the correct vertices, but the third vertex would extend far beyond the screen. I ended up replacing the original code to find the points for each triangle then make the recursive calls to `fractal()` instead. Unfortunately, I also realized rather late into the assignment that I had completely misrepresented the orientation of the base and recursive triangles. The Sierpinski triangle that our program should have produced has an upside-down parent triangle, with the child triangles (also upside-down) positioned along the left, right, and bottom corners of the base triangle, effectively creating the fractal from inside out. This realization came too late for me to drastically overhaul my code.

3.4 Source Code

Listing 11: Makefile

```
1 CC = g++
2 FLAGS = --std=c++20 -Wall -Werror -pedantic -g
3 LIBS = -lsfml-graphics -lsfml-window -lsfml-system
4
5 # .hpp files
6 DEPS = triangle.hpp
7
8 # .o files
9 OBJ = main.o triangle.o
10
11 # name of program
12 PROGRAM = Triangle
13
14 .PHONY: all clean lint
15
16 all: $(PROGRAM)
17
18 # wildcard recipe to compile all .cpp files in .o files
19 %.o: %.cpp $(DEPS)
20     $(CC) $(FLAGS) -c $<
21
22 $(PROGRAM): $(OBJ)
23     $(CC) $(FLAGS) -o $@ $^ $(LIBS)
24
25 clean:
26     rm -f *.o $(PROGRAM)
27
28 lint:
29     cpplint *.cpp *.hpp
```

Listing 12: main.cpp

```
1 // Copyright 2025
2 // Jayden Danquah
3
4 #include <iostream>
5 #include <cmath>
6 #include <SFML/Graphics.hpp>
7 #include "triangle.hpp"
8
9 int main(int argc, char* argv[]) {
10     if (argc != 3) {
11         std::cout << "Usage: ./triangle length depth" << std::endl;
12         return 1;
13     }
14     // take in length and depth from command line
15     double length = std::stod(argv[1]);
16     int depth = std::stoi(argv[2]);
17
18     // create window
19     sf::RenderWindow window(sf::VideoMode(800, 800), "Sierpinski Triangle");
20
21     // define base triangle points
22     sf::Vector2f top(400, 800);
23     sf::Vector2f left(0, 0);
24     sf::Vector2f right(800, 0);
25 }
```

```

26 // create triangle object and draw base triangle
27 Triangle triangle(length, depth);
28
29 // display window
30 while (window.isOpen()) {
31     sf::Event event;
32     while (window.pollEvent(event)) {
33         if (event.type == sf::Event::Closed) {
34             window.close();
35         }
36     }
37     window.clear();
38     triangle.base(window, top, left, right, depth);
39     window.display();
40 }
41 return 0;
42 }

```

Listing 13: Triangle.cpp

```

1 // Copyright 2025
2 // Jayden Danquah
3
4 #include "triangle.hpp"
5 #include <cmath>
6 #include <SFML/Graphics.hpp>
7
8 // constructor to take in length and depth from command line
9 Triangle::Triangle(double length, int depth) {
10     this->length = length;
11     this->depth = depth;
12 }
13
14 // function to draw base triangle
15 void Triangle::base(sf::RenderTarget& window, sf::Vector2f top,
16     sf::Vector2f left, sf::Vector2f right, int depth) {
17     sf::ConvexShape triangle;
18     triangle.setPointCount(3);
19     triangle.setPoint(0, top);
20     triangle.setPoint(1, left);
21     triangle.setPoint(2, right);
22     triangle.setFillColor(sf::Color::Transparent);
23     triangle.setOutlineColor(sf::Color::Blue);
24     triangle.setOutlineThickness(2);
25     window.draw(triangle);
26
27     // call to helper function
28     fractal(window, top, left, right, depth);
29 }
30
31 // helper function to draw smaller triangles
32 void Triangle::fractal(sf::RenderTarget& window, sf::Vector2f top,
33     sf::Vector2f left, sf::Vector2f right, int depth) {
34     if (depth <= 0) {
35         sf::ConvexShape triangle(3);
36         triangle.setOutlineColor(sf::Color::White);
37         triangle.setOutlineThickness(2);
38         triangle.setFillColor(sf::Color::Cyan);
39         triangle.setPoint(0, top);
40         triangle.setPoint(1, left);

```

```

41     triangle.setPoint(2, right);
42     window.draw(triangle);
43 } else {
44     // find midpoints of each side
45     sf::Vector2f leftMid((top.x + left.x) / 2.0f,
46     (top.y + left.y) / 2.0f);
47     sf::Vector2f rightMid((top.x + right.x) / 2.0f,
48     (top.y + right.y) / 2.0f);
49     sf::Vector2f bottomMid((left.x + right.x) / 2.0f,
50     (left.y + right.y) / 2.0f);
51
52     // recursive calls for smaller triangles
53     fractal(window, top, leftMid, rightMid, depth - 1);
54     fractal(window, leftMid, left, bottomMid, depth - 1);
55     fractal(window, rightMid, bottomMid, right, depth - 1);
56 }
57 }

```

Listing 14: Triangle.hpp

```

1  // Copyright 2025
2  // Jayden Danquah
3
4  #pragma once
5  #ifndef TRIANGLE_HPP
6  #define TRIANGLE_HPP
7
8  #include <SFML/Graphics.hpp>
9
10 class Triangle {
11 public:
12     // constructor to take in length and depth from command line
13     Triangle(double length, int depth);
14
15     // function to draw base triangle
16     void base(sf::RenderTarget& window, sf::Vector2f top,
17     sf::Vector2f left, sf::Vector2f right, int depth);
18
19     // helper function to draw smaller triangles
20     void fractal(sf::RenderTarget& window, sf::Vector2f top,
21     sf::Vector2f left, sf::Vector2f right, int depth);
22
23 private:
24     // length of one side of base equilateral triangle
25     double length;
26
27     // depth of recursion
28     int depth;
29 };
30 #endif

```

4 PS3: N-Body Simulation

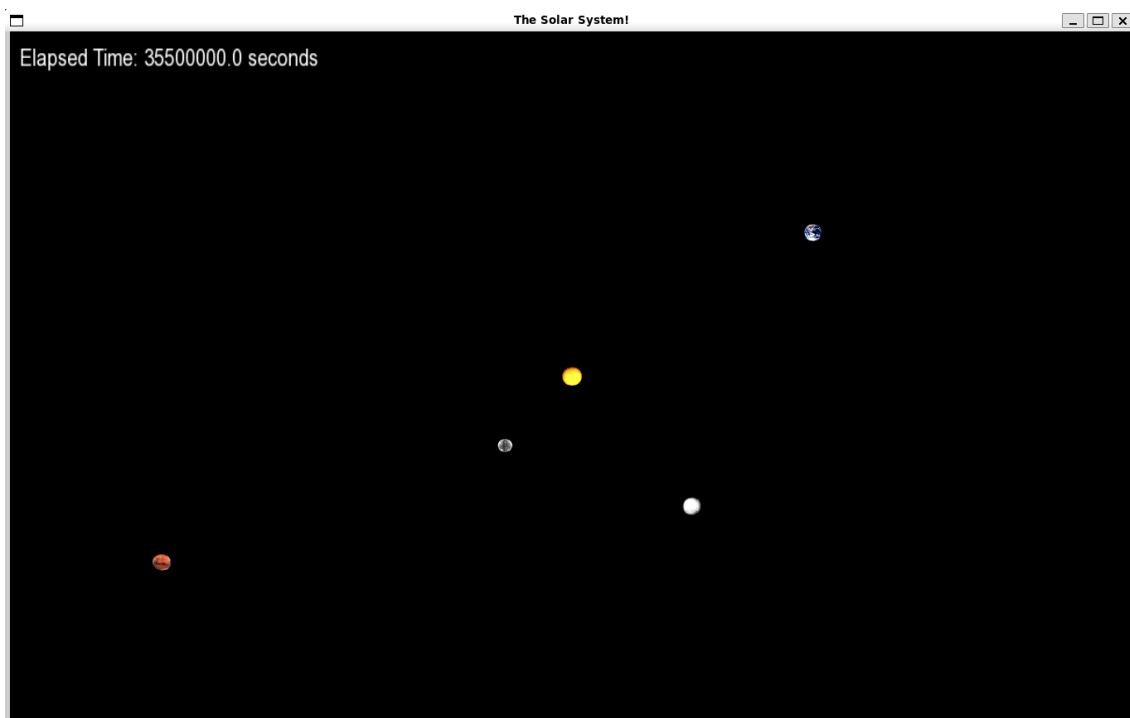


Figure 5: Window produced from running the code

4.1 Assignment Overview

In this two-part assignment, our objective was to create an N-Body simulation, animating a system of particles under the influence of mutual gravitational forces. The first part tasked us with loading a static image of our simulation (themed after the planets in the solar system) based on an input file containing data such as the size and radius of the universe, as well as the position, velocity, and .gif files for each of the planets being inserted. In the second part, we added the actual physics and animation to our simulation.

4.2 My Coding Approach

My implementation involved two classes, `CelestialBody` and `Universe`. The `CelestialBody` class contains variables and functions that retrieve the .gif files associated with the planet names passed into the object, insert them into a sprite, and render them to the window. Accessor and mutator functions were developed for both classes' overloaded operators, parsing in the planet's coordinates and filenames for easier access when coding each classes' drawing function. The `Universe` class assembles a vector of `CelestialBody` objects, which is utilized to output data from the input file to the terminal while also enabling the display of each planet on the screen. To introduce real-time motion into the simulation in the second part of the assignment, the `CelestialBody` class was updated to include methods of tracking the velocity and position of each planet, with the `update_position()` method adjusting the visual representation of the planets on screen by flipping the y-coordinate, accounting for the coordinate system used by the SFML library. In the `Universe` class, a newly added `step()` function iterates over every possible pair of planets in the vector, calculates their gravitational force and distance based on the pairwise formula, and subsequently updates their positions and velocities. For extra credit, I added an elapsed time counter to the top left corner of the screen.

4.3 What I Learned and Other Reflections

Getting my simulation to display and animate was a challenge, since I was working with real-life physics while also adhering to the pixel unit standard that the SFML library uses. I ran into several roadblocks while creating my static universe image, much of which was caused by redundant scaling across my `CelestialBody` and `Universe` classes as well as the fact that I did not initially use smart pointers when loading in each planet.

4.4 Source Code

Listing 15: Makefile

```
1 CC = g++
2 FLAGS = -Wall -Werror -std=c++17 -pedantic -g
3 LIBS = -lsfml-graphics -lsfml-window -lsfml-system -
        lboost_unit_test_framework
4
5 # .hpp files
6 DEPS = Universe.hpp CelestialBody.hpp
7 # Compiled .o files
8 OBJ = Universe.o CelestialBody.o
9 # Name of the program
10 PROGRAM = NBody
11
12 .PHONY: all clean lint
13
14 all: $(PROGRAM) NBody.a test
15
16 %.o: %.cpp $(DEPS)
17     $(CC) $(FLAGS) -c $< -o $@
18
19 main.o: main.cpp $(DEPS)
20     $(CC) $(FLAGS) -c main.cpp -o main.o
21
22 test.o: test.cpp $(DEPS)
23     $(CC) $(FLAGS) -c test.cpp -o test.o
24
25 $(PROGRAM): main.o $(OBJ)
26     $(CC) $(FLAGS) -o $@ $^ $(LIBS)
27
28 test: test.o $(OBJ)
29     $(CC) $(FLAGS) -o $@ $^ $(LIBS)
30
31 NBody.a: $(OBJ)
32     ar rcs $@ $^
33
34 clean:
35     rm -f *.o $(PROGRAM) test NBody.a
36
37 lint:
38     cpplint *.cpp *.hpp
```

Listing 16: main.cpp

```
1 // Copyright 2025
2 // by Jayden Danquah
3
4 #include <iostream>
5 #include <string>
6 #include <vector>
7 #include <iomanip>
8 #include <sstream>
9 #include <SFML/Graphics.hpp>
10 #include "Universe.hpp"
11 #include "CelestialBody.hpp"
12
13 int main(int argc, char* argv[]) {
14     // read values from command line
15     if (argc != 3) {
```

```

16     std::cerr << "Usage: " << argv[0] << " <T> <t>" << std::endl;
17     return 1;
18 }
19 double T = std::stod(argv[1]);
20 double dt = std::stod(argv[2]);
21
22 // create SFML window
23 sf::RenderWindow window(sf::VideoMode(1200, 600), "The Solar System!");
24
25 // clock, time, and text variables to track time
26 sf::Clock clock;
27 double elapsed_time = 0.0;
28 sf::Font font;
29 if (!font.loadFromFile("Arial.ttf")) {
30     std::cerr << "ERROR - failed to load font" << std::endl;
31     return 1;
32 }
33 sf::Text time;
34 time.setFont(font);
35 time.setCharacterSize(20);
36 time.setFillColor(sf::Color::White);
37 time.setPosition(10.f, 10.f);
38
39 // construct universe and resize window
40 NB::Universe uni;
41 std::cin >> uni;
42 uni.set_window_size(window.getSize());
43
44 // main draw loop
45 while (window.isOpen() && elapsed_time < T) {
46     sf::Event event;
47     while (window.pollEvent(event)) {
48         if (event.type == sf::Event::Closed)
49             window.close();
50     }
51
52     // compute time step and update elapsed time
53     uni.step(dt);
54     elapsed_time += dt;
55     std::ostringstream ss;
56     ss << std::fixed << std::setprecision(1) << elapsed_time;
57     time.setString("Elapsed Time: " + ss.str() + " seconds");
58
59     // draw universe and elapsed time to screen
60     window.clear(sf::Color::Black);
61     window.draw(uni);
62     window.setView(window.getDefaultView());
63     window.draw(time);
64     window.display();
65 }
66
67 // print universe data to terminal
68 std::cout << uni;
69 return 0;
70 }

```

Listing 17: CelestialBody.cpp

```

1 // Copyright 2025
2 // by Jayden Danquah

```

```

3
4 #include <iostream>
5 #include <string>
6 #include <memory>
7 #include <SFML/Graphics.hpp>
8 #include "CelestialBody.hpp"
9
10 // default constructor
11 NB::CelestialBody::CelestialBody(): x_pos(0), y_pos(0),
12     x_vel(0), y_vel(0), _mass(0), name(""), isTextureLoaded(false) {}
13
14 // overloaded input operator
15 std::istream& NB::operator>>(std::istream& is, NB::CelestialBody& cb) {
16     double x, y, vx, vy, m;
17     std::string name;
18
19     // read in numerical values
20     if (!(is >> x >> y >> vx >> vy >> m)) {
21         return is;
22     }
23
24     // read in filename and trim whitespace
25     std::getline(is >> std::ws, name);
26     name.erase(0, name.find_first_not_of(" \t"));
27     name.erase(name.find_last_not_of(" \t") + 1);
28
29     // assign values
30     cb.set_x_pos(x);
31     cb.set_y_pos(y);
32     cb.set_x_vel(vx);
33     cb.set_y_vel(vy);
34     cb.set_mass(m);
35     cb.set_name(name);
36     return is;
37 }
38
39 // load and scales texture for celestial body into sprite
40 void NB::CelestialBody::set_name(const std::string& n) {
41     // get filepath
42     name = n;
43
44     // create smart pointer to texture
45     body = std::make_shared<sf::Texture>();
46
47     // load in texture
48     if (!(*body).loadFromFile(name)) {
49         std::cerr << "ERROR - failed to load texture: "
50             << name << std::endl;
51         isTextureLoaded = false;
52     } else {
53         isTextureLoaded = true;
54         sprite.setTexture(*body);
55         sprite.setOrigin((*body).getSize().x / 2.0f,
56             (*body).getSize().y / 2.0f);
57         // scale factor
58         sprite.setScale(
59             2.50e+11 * 2 / 1200,
60             2.50e+11 * 2 / 800);
61         sprite.setPosition(x_pos, y_pos);

```

```

62     }
63 }
64
65 // draws celestial body sprite to window
66 void NB::CelestialBody::draw(sf::RenderTarget& window,
67     sf::RenderStates states) const {
68     // draw placeholder in case texture loading fails
69     if (!isTextureLoaded) {
70         sf::CircleShape placeholder(20.f);
71         placeholder.setFillColor(sf::Color::Red);
72         placeholder.setPosition(x_pos, y_pos);
73         window.draw(placeholder, states);
74         return;
75     }
76     window.draw(sprite, states);
77 }
78
79 // overloaded output operator
80 std::ostream& NB::operator<<(std::ostream& os, const NB::CelestialBody& cb)
81 {
82     os << cb.position().x << " " << cb.position().y << " "
83     << cb.velocity().x << " " << cb.velocity().y << " "
84     << cb.mass() << " " << cb.get_name();
85     return os;
86 }
87
88 // update velocity based on acceleration and time step
89 void NB::CelestialBody::update_velocity(double ax,
90     double ay, double dt) {
91     x_vel += ax * dt;
92     y_vel += ay * dt;
93 }
94
95 // update position based on velocity and time step
96 void NB::CelestialBody::update_position(double dt) {
97     x_pos += x_vel * dt;
98     y_pos += y_vel * dt;
99     sprite.setPosition(x_pos, -y_pos);
100 }

```

Listing 18: CelestialBody.hpp

```

1 // Copyright 2025
2 // by Jayden Danquah
3
4 #pragma once
5
6 #include <iostream>
7 #include <string>
8 #include <memory>
9 #include <SFML/Graphics.hpp>
10
11 namespace NB {
12 class CelestialBody: public sf::Drawable {
13 public:
14     explicit CelestialBody();
15
16     // mutators
17     void set_x_pos(double x) { x_pos = x; }
18     void set_y_pos(double y) { y_pos = y; }

```

```

19 void set_x_vel(double vx) { x_vel = vx; }
20 void set_y_vel(double vy) { y_vel = vy; }
21 void set_mass(double m) { _mass = m; }
22 void set_name(const std::string& n);
23
24 // physics
25 void update_velocity(double ax, double ay, double dt);
26 void update_position(double dt);
27
28 // accessors
29 sf::Vector2f position() const {
30     return sf::Vector2f(static_cast<float>(x_pos),
31         static_cast<float>(y_pos));
32 }
33 sf::Vector2f velocity() const {
34     return sf::Vector2f(static_cast<float>(x_vel),
35         static_cast<float>(y_vel));
36 }
37 double mass() const { return _mass; }
38 const std::string& get_name() const { return name; }
39
40 protected:
41     void draw(sf::RenderTarget& window,
42         sf::RenderStates states) const override;
43
44 private:
45     // position
46     double x_pos, y_pos;
47     // velocity
48     double x_vel, y_vel;
49     // mass
50     double _mass;
51     std::string name;
52     // sprite for rendering body
53     sf::Sprite sprite;
54     // texture for body
55     std::shared_ptr<sf::Texture> body;
56     bool isTextureLoaded = false;
57 };
58 std::istream& operator>>(std::istream& is, CelestialBody& cb);
59 std::ostream& operator<<(std::ostream& os, const CelestialBody& cb);
60 } // namespace NB

```

Listing 19: Universe.cpp

```

1 // Copyright 2025
2 // by Jayden Danquah
3
4 #include <iostream>
5 #include <vector>
6 #include <cmath>
7 #include "Universe.hpp"
8 #include "CelestialBody.hpp"
9
10 // default constructor
11 NB::Universe::Universe() : size(0), radius(0) {}
12
13 // overloaded input operator
14 std::istream& NB::operator>>(std::istream& is, NB::Universe& uni) {
15     // read values into local variables

```

```

16     size_t s;
17     double r;
18     if (!(is >> s >> r)) {
19         return is;
20     }
21     uni.set_radius(r);
22
23     // read in celestial bodies
24     size_t body_count = 0;
25     for (size_t i = 0; i < s; i++) {
26         NB::CelestialBody body;
27         if (!(is >> body)) {
28             break;
29         }
30         uni.add_body(body);
31         body_count++;
32     }
33     uni.set_size(body_count);
34     return is;
35 }
36
37 // draws all celestial bodies in universe to window
38 void NB::Universe::draw(sf::RenderTarget& window,
39 sf::RenderStates states) const {
40     sf::View view;
41     double viewSize = 2 * radius;
42     view.setSize(viewSize, viewSize);
43     view.setCenter(0, 0);
44     window.setView(view);
45
46     for (const auto& body : bodies) {
47         window.draw(body, states);
48     }
49 }
50
51 // overloaded output operator
52 std::ostream& NB::operator<<(std::ostream& os, const NB::Universe& uni) {
53     // print universe radius and size
54     os << uni.get_size() << "\n" << uni.get_radius() << std::endl;
55
56     // output celestial body data
57     for (const auto& body : uni.get_bodies()) {
58         os << body << std::endl;
59     }
60
61     return os;
62 }
63
64 void NB::Universe::step(double dt) {
65     // gravitational constant for calculations
66     const double G = 6.67e-11;
67
68     // initialize and calculate net force on each body
69     std::vector<std::pair<double, double>> forces(
70         bodies.size(), {0.0, 0.0});
71
72     // loop through all pairs of bodies in universe
73     for (auto i = bodies.begin(); i != bodies.end(); i++) {
74         // find position of first body

```

```

75     sf::Vector2f pos_i = (*i).position();
76     double x_i = pos_i.x;
77     double y_i = pos_i.y;
78
79     // loop through remaining bodies
80     for (auto j = i + 1; j != bodies.end(); j++) {
81         // find position of second body
82         sf::Vector2f pos_j = (*j).position();
83         double x_j = pos_j.x;
84         double y_j = pos_j.y;
85
86         // calculate distance between bodies
87         double dx = x_j - x_i;
88         double dy = y_j - y_i;
89         double dist = std::sqrt(dx * dx + dy * dy);
90
91         // prevent division by 0
92         if (dist == 0) {
93             continue;
94         }
95
96         // find force between bodies
97         double force = G * (*i).mass() * (*j).mass() /
98             (dist * dist);
99
100        // calculate x and y force components
101        double fx = force * dx / dist;
102        double fy = force * dy / dist;
103
104        // find indexes of bodies
105        size_t index_i = std::distance(bodies.begin(), i);
106        size_t index_j = std::distance(bodies.begin(), j);
107
108        // apply forces to pairs
109        forces[index_i].first += fx;
110        forces[index_i].second += fy;
111        forces[index_j].first -= fx;
112        forces[index_j].second -= fy;
113    }
114 }
115
116 // update velocity of each body
117 for (auto i = bodies.begin(); i != bodies.end(); i++) {
118     size_t index_i = std::distance(bodies.begin(), i);
119     double ax = forces[index_i].first / (*i).mass();
120     double ay = forces[index_i].second / (*i).mass();
121     (*i).update_velocity(ax, ay, dt);
122 }
123
124 // go back and update positions using new velocities
125 for (auto i = bodies.begin(); i != bodies.end(); i++) {
126     (*i).update_position(dt);
127 }
128 }

```

Listing 20: Universe.hpp

```

1 // Copyright 2025
2 // by Jayden Danquah
3

```

```

4  #pragma once
5
6  #include <iostream>
7  #include <vector>
8  #include <SFML/Graphics.hpp>
9  #include "CelestialBody.hpp"
10
11 namespace NB {
12 class Universe: public sf::Drawable {
13 public:
14     Universe();
15     void step(double dt);
16
17     // mutators
18     void set_size(size_t s) { size = s; }
19     void set_radius(double r) { radius = r; }
20     void set_window_size(const sf::Vector2u& ws) { window_size = ws; }
21     void add_body(const CelestialBody& body) { bodies.push_back(body); }
22
23     // accessors
24     size_t get_size() const { return size; }
25     double get_radius() const { return radius; }
26     sf::Vector2u get_window_size() const { return window_size; }
27     const std::vector<CelestialBody>& get_bodies() const { return bodies; }
28     const CelestialBody& operator[](size_t i) const { return bodies[i]; }
29
30 protected:
31     void draw(sf::RenderTarget& window,
32             sf::RenderStates states) const override;
33
34 private:
35     size_t size;
36     double radius;
37     sf::Vector2u window_size;
38     std::vector<CelestialBody> bodies;
39 };
40 std::istream& operator>>(std::istream& is, Universe& uni);
41 std::ostream& operator<<(std::ostream& os, const Universe& uni);
42 } // namespace NB

```

Listing 21: test.cpp

```

1  // Copyright 2025
2  // by Jayden Danquah
3
4  #include <string>
5  #include <fstream>
6  #include <sstream>
7  #include "Universe.hpp"
8  #include "CelestialBody.hpp"
9
10 #define BOOST_TEST_DYN_LINK
11 #define BOOST_TEST_MODULE Main
12 #include <boost/test/unit_test.hpp>
13
14 using NB::Universe;
15 using NB::CelestialBody;
16
17 // test that celestial body inserts and extracts as intended
18 BOOST_AUTO_TEST_CASE(test_celestial_body) {

```

```

19     NB::CelestialBody body;
20     std::istringstream input("0.0 0.0 0.0 0.0 0.0 earth.gif");
21     input >> body;
22     std::ostringstream output;
23     output << body;
24
25     BOOST_CHECK_EQUAL(body.position().x, 0.0f);
26     BOOST_CHECK_EQUAL(body.position().y, 0.0f);
27     BOOST_CHECK_EQUAL(body.velocity().x, 0.0f);
28     BOOST_CHECK_EQUAL(body.velocity().y, 0.0f);
29     BOOST_CHECK_EQUAL(body.mass(), 0.0f);
30
31     std::string expected_output = "0 0 0 0 0 earth.gif";
32     BOOST_CHECK_EQUAL(output.str(), expected_output);
33 }
34
35 // no step
36 BOOST_AUTO_TEST_CASE(test_no_step) {
37     Universe uni;
38     std::ifstream input("planets.txt");
39     input >> uni;
40     uni.step(0.0);
41
42     // check difference in earth values pre- and post-step
43     BOOST_CHECK_CLOSE(uni[0].position().x, 1.4960e+11, 0.1);
44     BOOST_CHECK_CLOSE(uni[0].position().y, 0.0000e+00, 0.1);
45     BOOST_CHECK_CLOSE(uni[0].velocity().x, 0.0000e+00, 0.1);
46     BOOST_CHECK_CLOSE(uni[0].velocity().y, 2.9800e+04, 0.1);
47
48     // check difference in mars values pre- and post-step
49     BOOST_CHECK_CLOSE(uni[1].position().x, 2.2790e+11, 0.1);
50     BOOST_CHECK_CLOSE(uni[1].position().y, 0.0000e+00, 0.1);
51     BOOST_CHECK_CLOSE(uni[1].velocity().x, 0.0000e+00, 0.1);
52     BOOST_CHECK_CLOSE(uni[1].velocity().y, 2.4100e+04, 0.1);
53
54     // check difference in mercury values pre- and post-step
55     BOOST_CHECK_CLOSE(uni[2].position().x, 5.7900e+10, 0.1);
56     BOOST_CHECK_CLOSE(uni[2].position().y, 0.0000e+00, 0.1);
57     BOOST_CHECK_CLOSE(uni[2].velocity().x, 0.0000e+00, 0.1);
58     BOOST_CHECK_CLOSE(uni[2].velocity().y, 4.7900e+04, 0.1);
59
60     // check difference in sun values pre- and post-step
61     BOOST_CHECK_CLOSE(uni[3].position().x, 0.0000e+00, 0.1);
62     BOOST_CHECK_CLOSE(uni[3].position().y, 0.0000e+00, 0.1);
63     BOOST_CHECK_CLOSE(uni[3].velocity().x, 0.0000e+00, 0.1);
64     BOOST_CHECK_CLOSE(uni[3].velocity().y, 0.0000e+00, 0.1);
65
66     // check difference in venus values pre- and post-step
67     BOOST_CHECK_CLOSE(uni[4].position().x, 1.0820e+11, 0.1);
68     BOOST_CHECK_CLOSE(uni[4].position().y, 0.0000e+00, 0.1);
69     BOOST_CHECK_CLOSE(uni[4].velocity().x, 0.0000e+00, 0.1);
70     BOOST_CHECK_CLOSE(uni[4].velocity().y, 3.5000e+04, 0.1);
71 }
72
73 // one step
74 BOOST_AUTO_TEST_CASE(test_one_step) {
75     Universe uni;
76     std::ifstream input("planets.txt");
77     input >> uni;

```

```

78
79 // perform one step
80 uni.step(25000.0);
81
82 // check difference in earth values pre- and post-step
83 BOOST_CHECK_CLOSE(uni[0].position().x, 1.4960e+11, 0.1);
84 BOOST_CHECK_CLOSE(uni[0].position().y, 7.4500e+08, 0.1);
85 BOOST_CHECK_CLOSE(uni[0].velocity().x, -1.4820e+02, 0.1);
86 BOOST_CHECK_CLOSE(uni[0].velocity().y, 2.9800e+04, 0.1);
87
88 // check difference in mars values pre- and post-step
89 BOOST_CHECK_CLOSE(uni[1].position().x, 2.2790e+11, 0.1);
90 BOOST_CHECK_CLOSE(uni[1].position().y, 6.0250e+08, 0.1);
91 BOOST_CHECK_CLOSE(uni[1].velocity().x, -6.3860e+01, 0.1);
92 BOOST_CHECK_CLOSE(uni[1].velocity().y, 2.4100e+04, 0.1);
93
94 // check difference in mercury values pre- and post-step
95 BOOST_CHECK_CLOSE(uni[2].position().x, 5.7875e+10, 0.1);
96 BOOST_CHECK_CLOSE(uni[2].position().y, 1.1975e+09, 0.1);
97 BOOST_CHECK_CLOSE(uni[2].velocity().x, -9.8933e+02, 0.1);
98 BOOST_CHECK_CLOSE(uni[2].velocity().y, 4.7900e+04, 0.1);
99
100 // check difference in sun values pre- and post-step
101 BOOST_CHECK_CLOSE(uni[3].position().x, 3.3087e+01, 10.0);
102 BOOST_CHECK_CLOSE(uni[3].position().y, 0.0000e+00, 10.0);
103 BOOST_CHECK_CLOSE(uni[3].velocity().x, 1.3235e-03, 10.0);
104 BOOST_CHECK_CLOSE(uni[3].velocity().y, 0.0000e+00, 0.1);
105
106 // check difference in venus values pre- and post-step
107 BOOST_CHECK_CLOSE(uni[4].position().x, 1.0819e+11, 0.1);
108 BOOST_CHECK_CLOSE(uni[4].position().y, 8.7500e+08, 0.1);
109 BOOST_CHECK_CLOSE(uni[4].velocity().x, -2.8329e+02, 0.1);
110 BOOST_CHECK_CLOSE(uni[4].velocity().y, 3.5000e+04, 0.1);
111 }
112
113 // two steps
114 BOOST_AUTO_TEST_CASE(test_two_step) {
115     Universe uni;
116     std::ifstream input("planets.txt");
117     input >> uni;
118
119     // perform two steps
120     uni.step(25000.0);
121     uni.step(25000.0);
122
123     // check difference in earth values pre- and post-step
124     BOOST_CHECK_CLOSE(uni[0].position().x, 1.4959e+11, 0.1);
125     BOOST_CHECK_CLOSE(uni[0].position().y, 1.4900e+09, 0.1);
126     BOOST_CHECK_CLOSE(uni[0].velocity().x, -2.9640e+02, 0.1);
127     BOOST_CHECK_CLOSE(uni[0].velocity().y, 2.9799e+04, 0.1);
128
129     // check difference in mars values pre- and post-step
130     BOOST_CHECK_CLOSE(uni[1].position().x, 2.2790e+11, 0.1);
131     BOOST_CHECK_CLOSE(uni[1].position().y, 1.2050e+09, 0.1);
132     BOOST_CHECK_CLOSE(uni[1].velocity().x, -1.2772e+02, 0.1);
133     BOOST_CHECK_CLOSE(uni[1].velocity().y, 2.4100e+04, 0.1);
134
135     // check difference in mercury values pre- and post-step
136     BOOST_CHECK_CLOSE(uni[2].position().x, 5.7826e+10, 0.1);

```

```

137 BOOST_CHECK_CLOSE(uni[2].position().y, 2.3945e+09, 0.1);
138 BOOST_CHECK_CLOSE(uni[2].velocity().x, -1.9789e+03, 0.1);
139 BOOST_CHECK_CLOSE(uni[2].velocity().y, 4.7880e+04, 0.1);
140
141 // check difference in sun-values pre- and post-step
142 BOOST_CHECK_CLOSE(uni[3].position().x, 9.9262e+01, 10.0);
143 BOOST_CHECK_CLOSE(uni[3].position().y, 2.8198e-01, 10.0);
144 BOOST_CHECK_CLOSE(uni[3].velocity().x, 2.6470e-03, 10.0);
145 BOOST_CHECK_CLOSE(uni[3].velocity().y, 1.1279e-05, 10.0);
146
147 // check difference in venus values pre- and post-step
148 BOOST_CHECK_CLOSE(uni[4].position().x, 1.0818e+11, 0.1);
149 BOOST_CHECK_CLOSE(uni[4].position().y, 1.7499e+09, 0.1);
150 BOOST_CHECK_CLOSE(uni[4].velocity().x, -5.6660e+02, 0.1);
151 BOOST_CHECK_CLOSE(uni[4].velocity().y, 3.4998e+04, 0.1);
152 }
153
154 // three steps
155 BOOST_AUTO_TEST_CASE(test_three_step) {
156     Universe uni;
157     std::ifstream input("planets.txt");
158     input >> uni;
159
160     // perform three steps
161     uni.step(25000.0);
162     uni.step(25000.0);
163     uni.step(25000.0);
164
165     // check difference in earth values pre- and post-step
166     BOOST_CHECK_CLOSE(uni[0].position().x, 1.4958e+11, 0.1);
167     BOOST_CHECK_CLOSE(uni[0].position().y, 2.2349e+09, 0.1);
168     BOOST_CHECK_CLOSE(uni[0].velocity().x, -4.4460e+02, 0.1);
169     BOOST_CHECK_CLOSE(uni[0].velocity().y, 2.9798e+04, 0.1);
170
171     // check difference in mars values pre- and post-step
172     BOOST_CHECK_CLOSE(uni[1].position().x, 2.2789e+11, 0.1);
173     BOOST_CHECK_CLOSE(uni[1].position().y, 1.8075e+09, 0.1);
174     BOOST_CHECK_CLOSE(uni[1].velocity().x, -1.9158e+02, 0.1);
175     BOOST_CHECK_CLOSE(uni[1].velocity().y, 2.4099e+04, 0.1);
176
177     // check difference in mercury values pre- and post-step
178     BOOST_CHECK_CLOSE(uni[2].position().x, 5.7752e+10, 0.1);
179     BOOST_CHECK_CLOSE(uni[2].position().y, 3.5905e+09, 0.1);
180     BOOST_CHECK_CLOSE(uni[2].velocity().x, -2.9682e+03, 0.1);
181     BOOST_CHECK_CLOSE(uni[2].velocity().y, 4.7839e+04, 0.1);
182
183     // check difference in sun-values pre- and post-step
184     BOOST_CHECK_CLOSE(uni[3].position().x, 1.9852e+02, 10.0);
185     BOOST_CHECK_CLOSE(uni[3].position().y, 1.1280e+00, 10.0);
186     BOOST_CHECK_CLOSE(uni[3].velocity().x, 3.9705e-03, 10.0);
187     BOOST_CHECK_CLOSE(uni[3].velocity().y, 3.3841e-05, 10.0);
188
189     // check difference in venus values pre- and post-step
190     BOOST_CHECK_CLOSE(uni[4].position().x, 1.0816e+11, 0.1);
191     BOOST_CHECK_CLOSE(uni[4].position().y, 2.6248e+09, 0.1);
192     BOOST_CHECK_CLOSE(uni[4].velocity().x, -8.4989e+02, 0.1);
193     BOOST_CHECK_CLOSE(uni[4].velocity().y, 3.4993e+04, 0.1);
194 }

```

5 PS4: Sokoban



Figure 6: Start of the game

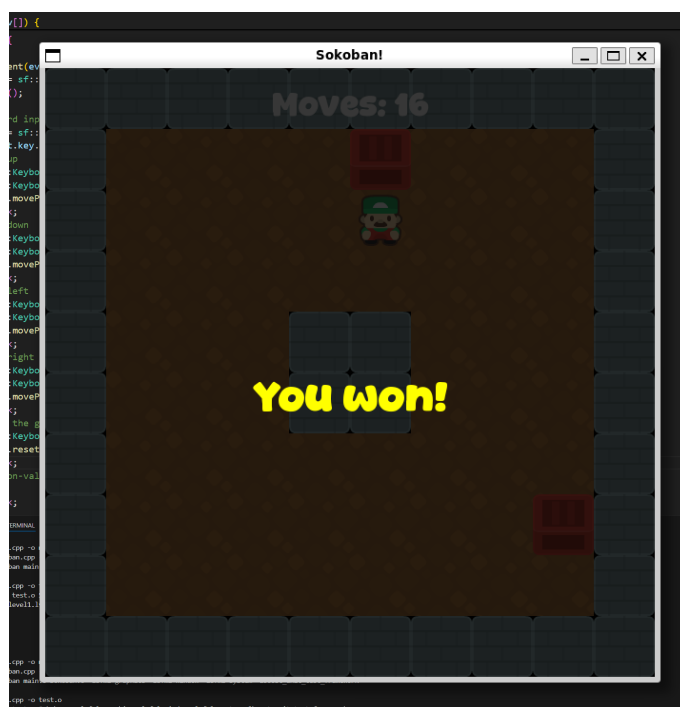


Figure 7: End of the game

5.1 Assignment Overview

This two-part assignment had us recreate Sokoban, a Japanese tile-based puzzle game in which the player pushes boxes around a warehouse to get them in their designated storage locations. Like the N-Body Simulator, the first part of the assignment had us create a static image based on a text file containing all of the level data, including the size of the grid and special characters mapped to all of the game's elements, such as the player and their initial location, the ground and wall tiles, the boxes, and the storage locations. The second part had us add the game mechanics.

5.2 My Coding Approach

The Sokoban class I created in the first part of the assignment reads the data from the level file and stores it inside a two-dimensional grid/matrix (represented as a vector of characters).

From there, I loaded the necessary .PNG files into sprites representing the player, the wall and ground tiles, the storage locations, and the boxes. To draw the level to the window, I utilized a switch statement with cases mapped to each special character in the text file; the wall and ground tiles are drawn first, with another layer drawn on top to draw the player, boxes, and locations. The two-layer approach was not something I immediately thought of when first coding, but I ultimately used it once I realized drawing the unique sprites to the window would have them surrounded by a black box caused by the absence of a ground tile. Methods such as `width()`, `height()`, and `playerLoc()` were used to return the level file's data to the console after the window was closed. For extra credit, I added a move counter to the top of the screen.

The game mechanics I added to the second part of the assignment included `reset()`, which restores the level back to its original state using a copy of the original grid data, and `isWon()`, a Boolean method that parses through the grid and checks if every box has been moved into a storage location. The main mechanic, `movePlayer()`, involved manipulating the grid data based on keyboard commands to make the sprites in the window move. When coding this function, it was important that I took into account every possible situation the user could find themselves in, such as attempting to move the player into a wall or having them push a box into another only for their movement to be blocked. I mostly handled this by using a reference to the next tile to determine the appropriate behavior, but even after the player or boxes moved, I also ensured that the previous tiles they were on were correctly restored.

5.3 What I Learned and Other Reflections

Of all the assignments I completed in this course, Sokoban had to be my favorite because it was the first time I coded a game using C/C++. Working on the N-Body Simulation helped me properly load in textures and render sprites to the window, so that was not much of an issue this time around. Having to manipulate the level data to get actual motion on the screen was both challenging and fascinating, and it makes me wonder if similar games coded in C/C++ would follow similar rules.

5.4 Source Code

Listing 22: Makefile

```

1 CC = g++
2 FLAGS = -Wall -Werror -std=c++17 -pedantic -g
3 LIBS = -lsfml-graphics -lsfml-window -lsfml-system -
        lboost_unit_test_framework
4
5 # .hpp files
6 DEPS = Sokoban.hpp
7 # Compiled .o files
8 OBJ = Sokoban.o
9 # Name of the program
10 PROGRAM = Sokoban
11
12 .PHONY: all clean lint
13
14 all: $(PROGRAM) Sokoban.a test
15
16 %.o: %.cpp $(DEPS)
17     $(CC) $(FLAGS) -c $< -o $@
18
19 main.o: main.cpp $(DEPS)
20     $(CC) $(FLAGS) -c main.cpp -o main.o
21
22 $(PROGRAM): main.o $(OBJ)
23     $(CC) $(FLAGS) -o $@ $~ $(LIBS)
24
25 test: test.o $(OBJ)

```

```

26 $(CC) $(FLAGS) -o $@ $^ $(LIBS)
27
28 Sokoban.a: $(OBJ)
29     ar rcs $@ $^
30
31 clean:
32     rm -f *.o $(PROGRAM) Sokoban.a test
33
34 lint:
35     cpplint *.cpp *.hpp

```

Listing 23: main.cpp

```

1  // Copyright 2025 by Jayden Danquah
2
3  #include <iostream>
4  #include <fstream>
5  #include <SFML/Graphics.hpp>
6  #include "Sokoban.hpp"
7
8  int main(int argc, char* argv[]) {
9      // check for proper command line arguments
10     if (argc != 2) {
11         std::cerr << "Usage: " << argv[0] << " <level-file>";
12         return 1;
13     }
14
15     // open level file
16     std::ifstream level(argv[1]);
17     if (!level) {
18         std::cerr << "ERROR - " << argv[1] << " couldn't be found";
19         return 1;
20     }
21
22     // create game object, read level data, then close file
23     SB::Sokoban game;
24     level >> game;
25     level.close();
26
27     // draw game window
28     sf::RenderWindow window(sf::VideoMode(game.pixelWidth(),
29     game.pixelHeight()), "Sokoban!");
30     window.setPosition({0, 0});
31
32     while (window.isOpen()) {
33         sf::Event event;
34         while (window.pollEvent(event)) {
35             if (event.type == sf::Event::Closed) {
36                 window.close();
37             }
38             // handle keyboard input
39             if (event.type == sf::Event::KeyPressed) {
40                 switch (event.key.code) {
41                     // move up
42                     case sf::Keyboard::Up:
43                     case sf::Keyboard::W:
44                         game.movePlayer(SB::Direction::Up);
45                         break;
46                     // move down
47                     case sf::Keyboard::Down:

```

```

48         case sf::Keyboard::S:
49             game.movePlayer(SB::Direction::Down);
50             break;
51         // move left
52         case sf::Keyboard::Left:
53         case sf::Keyboard::A:
54             game.movePlayer(SB::Direction::Left);
55             break;
56         // move right
57         case sf::Keyboard::Right:
58         case sf::Keyboard::D:
59             game.movePlayer(SB::Direction::Right);
60             break;
61         // reset the game
62         case sf::Keyboard::R:
63             game.reset();
64             break;
65         // all non-valid key presses
66         default:
67             break;
68     }
69 }
70 }
71 window.clear();
72 window.draw(game);
73 window.display();
74 }
75
76 // output level data
77 std::cout << game;
78 std::cout << "Current player position: ("
79 << game.playerLoc().x << ", " << game.playerLoc().y << ")" << std::endl;
80
81 return 0;
82 }

```

Listing 24: Sokoban.cpp

```

1  // Copyright 2025 by Jayden Danquah
2
3  #include <iostream>
4  #include <vector>
5  #include <fstream>
6  #include <string>
7  #include <SFML/Graphics.hpp>
8  #include "Sokoban.hpp"
9
10 // default constructor
11 SB::Sokoban::Sokoban(): _height(0), _width(0), _moves(0) {
12     // preload media files into textures
13     _font.loadFromFile("Ezydraw.ttf");
14     _wall.loadFromFile("block_06.png");
15     _empty.loadFromFile("ground_01.png");
16     _box.loadFromFile("crate_03.png");
17     _storage.loadFromFile("ground_04.png");
18     _player.loadFromFile("player_05.png");
19 }
20
21 // constructor
22 SB::Sokoban::Sokoban(const std::string& filename): _moves(0) {

```

```

23 // preload media files into textures
24 _font.loadFromFile("Ezydraw.ttf");
25 _wall.loadFromFile("block_06.png");
26 _empty.loadFromFile("ground_01.png");
27 _box.loadFromFile("crate_03.png");
28 _storage.loadFromFile("ground_04.png");
29 _player.loadFromFile("player_05.png");
30
31 // open level file
32 std::ifstream level(filename);
33 if (!level) {
34     std::cerr << "ERROR - could not find " << filename << std::endl;
35     return;
36 }
37
38 // use >> to load in level data
39 level >> *this;
40
41 // store backup of level data for resetting
42 _grid_copy = _grid_data;
43 }
44
45 // parses grid data and creates UI
46 void SB::Sokoban::draw(sf::RenderTarget& target,
47     sf::RenderStates states) const {
48     // row-major order: rows, then columns
49     for (size_t y = 0; y < _height; y++) {
50         for (size_t x = 0; x < _width; x++) {
51             // sprites to hold textures
52             sf::Sprite tile_1, tile_2;
53
54             // used to parse each char
55             char c = _grid_data[y][x];
56
57             // tile_1 reserved specifically for ground tile
58             tile_1.setTexture(_empty);
59
60             // used to layer other game elements over ground tiles
61             bool draw_tile_2 = false;
62
63             // switch statement swaps char with designated sprite
64             switch (c) {
65                 // initial player position
66                 case '@':
67                     tile_2.setTexture(_player);
68                     draw_tile_2 = true;
69                     break;
70                 // empty space
71                 case '.':
72                     break;
73                 // wall
74                 case '#':
75                     tile_1.setTexture(_wall);
76                     break;
77                 // box
78                 case 'A':
79                     tile_2.setTexture(_box);
80                     draw_tile_2 = true;
81                     break;

```

```

82         // storage location
83         case 'a':
84             tile_1.setTexture(_storage);
85             break;
86         // box already in storage location
87         case '1':
88             tile_1.setTexture(_storage);
89             tile_2.setTexture(_box);
90             draw_tile_2 = true;
91             break;
92         default:
93             break;
94     }
95     // draw ground tiles
96     tile_1.setPosition(x * TILE_SIZE, y * TILE_SIZE);
97     target.draw(tile_1, states);
98
99     // layer special sprites
100    if (draw_tile_2) {
101        tile_2.setPosition(x * TILE_SIZE, y * TILE_SIZE);
102        target.draw(tile_2, states);
103    }
104 }
105 }
106 // draw move counter
107 sf::Text counter("Moves: " + std::to_string(_moves), _font);
108 // get width of text for centering
109 sf::FloatRect counter_width = counter.getLocalBounds();
110 // center horizontally (subtract half the width)
111 float centerX = (target.getSize().x - counter_width.width) / 2.0f;
112 counter.setPosition(centerX, 20);
113 target.draw(counter, states);
114
115 // display message if game is won
116 if (isWon()) {
117     // semi-transparent background
118     sf::RectangleShape background(sf::Vector2f(target.getSize().x,
119         target.getSize().y));
120     background.setFillColor(sf::Color(0, 0, 0, 200));
121
122     // winning message
123     sf::Text win("You won!", _font, 40);
124     win.setFillColor(sf::Color::Yellow);
125
126     // get width of text for centering
127     sf::FloatRect text_width = win.getLocalBounds();
128     // center horizontally (subtract half the width)
129     win.setPosition((target.getSize().x - text_width.width) / 2,
130         target.getSize().y / 2);
131
132     target.draw(background, states);
133     target.draw(win, states);
134 }
135 }
136
137 // return initial location of player
138 sf::Vector2u SB::Sokoban::playerLoc() const {
139     for (size_t y = 0; y < _height; y++) {
140         for (size_t x = 0; x < _width; x++) {

```

```

141         if (_grid_data[y][x] == '@') {
142             return {static_cast<unsigned>(x), static_cast<unsigned>(y)};
143         }
144     }
145 }
146 throw std::runtime_error("ERROR - unable to find player location");
147 }
148
149 // read in level data
150 std::istream& SB::operator>>(std::istream& in, Sokoban& s) {
151     // read in height and width
152     size_t h, w;
153     if (!(in >> h >> w)) {
154         throw std::runtime_error("ERROR - invalid height and width");
155     }
156     s.set_height(h);
157     s.set_width(w);
158
159     // read in grid data line by line
160     std::vector<std::string> g;
161     std::string line;
162     std::getline(in, line);
163
164     for (size_t i = 0; i < h; i++) {
165         if (!std::getline(in, line)) {
166             throw std::runtime_error("ERROR - invalid grid data");
167         }
168         g.push_back(line);
169     }
170     s.set_grid(g);
171
172     // store backup of level data for resetting
173     s.set_grid_copy(g);
174
175     return in;
176 }
177
178 // write map back to terminal
179 std::ostream& SB::operator<<(std::ostream& out, const Sokoban& s) {
180     out << s.height() << " " << s.width() << std::endl;
181
182     const std::vector<std::string>& grid = s.grid();
183     for (const auto& row : grid) {
184         out << row << std::endl;
185     }
186
187     return out;
188 }
189
190 // moves player
191 void SB::Sokoban::movePlayer(SB::Direction dir) {
192     // if the game's been won, don't do anything
193     if (isWon()) return;
194
195     // get current player position
196     sf::Vector2u position = playerLoc();
197
198     // determine next position based on keypress
199     sf::Vector2u next = position;

```

```

200     switch (dir) {
201         case Direction::Up:
202             if (position.y > 0)
203                 next.y--;
204             break;
205         case Direction::Down:
206             if (position.y < _height - 1)
207                 next.y++;
208             break;
209         case Direction::Left:
210             if (position.x > 0)
211                 next.x--;
212             break;
213         case Direction::Right:
214             if (position.x < _width - 1)
215                 next.x++;
216             break;
217     }
218
219     // get reference to tile player is moving toward
220     char& next_tile = _grid_data[next.y][next.x];
221
222     if (next_tile == '.' || next_tile == 'a') {
223         // move player to empty space or storage tile
224         _grid_data[next.y][next.x] = '@';
225
226         // restore previous tile
227         char& current_tile = _grid_data[position.y][position.x];
228         if (_grid_copy[position.y][position.x] == 'a') {
229             // restore to storage tile
230             current_tile = 'a';
231         } else {
232             // restore to ground tile
233             current_tile = '.';
234         }
235         _moves++;
236     } else if (next_tile == 'A' || next_tile == 'a') {
237         // get box position
238         sf::Vector2u box_position = next;
239         switch (dir) {
240             case Direction::Up:
241                 if (box_position.y > 0)
242                     box_position.y--;
243                 break;
244             case Direction::Down:
245                 if (box_position.y < _height - 1)
246                     box_position.y++;
247                 break;
248             case Direction::Left:
249                 if (box_position.x > 0)
250                     box_position.x--;
251                 break;
252             case Direction::Right:
253                 if (box_position.x < _width - 1)
254                     box_position.x++;
255                 break;
256         }
257
258         // get reference to tile where box will be pushed

```

```

259     char& next_box = _grid_data[box_position.y][box_position.x];
260
261     if (next_box == '.' || next_box == 'a') {
262         // push box to next tile
263         next_box = (next_box == 'a') ? '1' : 'A';
264
265         // move player to place where the box was
266         next_tile = '@';
267
268         // restore player's old position
269         char& current_tile = _grid_data[position.y][position.x];
270         current_tile = (current_tile == '@') ? '.' : ' ';
271
272         _moves++;
273     }
274 }
275 }
276
277 // determines if all boxes are in correct positions
278 bool SB::Sokoban::isWon() const {
279     // parse thorough grid data
280     for (size_t y = 0; y < _height; y++) {
281         for (size_t x = 0; x < _width; x++) {
282             // check if there's a box not on a storage tile
283             if (_grid_data[y][x] == 'A') {
284                 return false;
285             }
286         }
287     }
288     return true;
289 }
290
291 // reset level back to original state
292 void SB::Sokoban::reset() {
293     _grid_data = _grid_copy;
294     _moves = 0;
295 }

```

Listing 25: Sokoban.hpp

```

1  // Copyright 2025 by Jayden Danquah
2
3  #pragma once
4  #include <vector>
5  #include <string>
6  #include <SFML/Graphics.hpp>
7
8  namespace SB {
9      enum class Direction {
10          Up, Down, Left, Right
11      };
12
13      class Sokoban : public sf::Drawable {
14      public:
15          static const int TILE_SIZE = 64;
16
17          Sokoban();
18          explicit Sokoban(const std::string&);
19
20          // mutators

```

```

21     void set_height(size_t h) { _height = h; }
22     void set_width(size_t w) { _width = w; }
23     void set_grid(const std::vector<std::string>& g) { _grid_data = g; }
24     void set_grid_copy(const std::vector<std::string>& g) { _grid_copy = g;
    }
25
26     // accessors
27     unsigned int pixelHeight() const { return _height * TILE_SIZE; }
28     unsigned int pixelWidth() const { return _width * TILE_SIZE; }
29     unsigned int height() const { return _height; }
30     unsigned int width() const { return _width; }
31     sf::Vector2u playerLoc() const;
32     const std::vector<std::string>& grid() const { return _grid_data; }
33
34     // game mechanics
35     bool isWon() const;
36     void movePlayer(Direction dir);
37     void reset();
38
39 protected:
40     void draw(sf::RenderTarget& target, sf::RenderStates states) const
        override;
41
42 private:
43     // data
44     size_t _height;
45     size_t _width;
46     std::vector<std::string> _grid_data;
47     std::vector<std::string> _grid_copy;
48     int _moves;
49
50     // textures
51     sf::Font _font;
52     sf::Texture _wall;
53     sf::Texture _empty;
54     sf::Texture _box;
55     sf::Texture _storage;
56     sf::Texture _player;
57     std::vector<sf::Texture> _grid;
58 };
59 std::ostream& operator<<(std::ostream& out, const Sokoban& s);
60 std::istream& operator>>(std::istream& in, Sokoban& s);
61 } // namespace SB

```

Listing 26: test.cpp

```

1 // Copyright 2025 by Jayden Danquah
2
3 #include <string>
4 #include <fstream>
5 #include <sstream>
6 #include "Sokoban.hpp"
7
8 #define BOOST_TEST_DYN_LINK
9 #define BOOST_TEST_MODULE Main
10 #include <boost/test/unit_test.hpp>
11
12 using SB::Sokoban;
13
14 // test that level data is correctly inserted

```

```

15 BOOST_AUTO_TEST_CASE(test_insertion) {
16     std::ifstream level("level1.lvl");
17     SB::Sokoban game;
18     level >> game;
19
20     std::ostringstream output;
21     output << game;
22
23     std::string expected_output =
24         "10 10\n"
25         "#####\n"
26         "#...a...\n"
27         "#...A...\n"
28         "#.....#\n"
29         "#...##...\n"
30         "#...##...\n"
31         "#..@..A..\n"
32         "#.....a#\n"
33         "#.....#\n"
34         "#####\n";
35
36     BOOST_CHECK_EQUAL(output.str(), expected_output);
37 }
38
39 // test that level data is correctly extracted
40 BOOST_AUTO_TEST_CASE(test_extraction) {
41     std::ifstream level("level1.lvl");
42     SB::Sokoban game;
43     level >> game;
44
45     BOOST_CHECK_EQUAL(game.height(), 10);
46     BOOST_CHECK_EQUAL(game.width(), 10);
47
48     std::ostringstream output;
49     output << game;
50
51     std::string expected_output =
52         "10 10\n"
53         "#####\n"
54         "#...a...\n"
55         "#...A...\n"
56         "#.....#\n"
57         "#...##...\n"
58         "#...##...\n"
59         "#..@..A..\n"
60         "#.....a#\n"
61         "#.....#\n"
62         "#####\n";
63
64     BOOST_CHECK_EQUAL(output.str(), expected_output);
65
66     BOOST_CHECK_EQUAL(game.playerLoc().y, 6);
67     BOOST_CHECK_EQUAL(game.playerLoc().x, 3);
68 }
69
70 // test that the player can move
71 BOOST_AUTO_TEST_CASE(test_movePlayer) {
72     std::ifstream level("level1.lvl");
73     SB::Sokoban game;

```

```

74     level >> game;
75
76     // initial position
77     sf::Vector2u init_pos = game.playerLoc();
78
79     // move player up
80     game.movePlayer(SB::Direction::Up);
81
82     // new position
83     sf::Vector2u new_pos = game.playerLoc();
84
85     // check difference in x and y values
86     BOOST_CHECK_EQUAL(new_pos.x, init_pos.x);
87     BOOST_CHECK_EQUAL(new_pos.y, init_pos.y - 1);
88 }
89
90 // test that player doesn't phase through boxes
91 BOOST_AUTO_TEST_CASE(test_box_phase) {
92     std::ifstream level("level1.lvl");
93     SB::Sokoban game;
94     level >> game;
95
96     // initial position should be (6, 3)
97     sf::Vector2u init_pos = game.playerLoc();
98
99     // move player towards box
100    game.movePlayer(SB::Direction::Right);
101    game.movePlayer(SB::Direction::Right);
102    game.movePlayer(SB::Direction::Right);
103
104    // grab new position
105    sf::Vector2u new_pos = game.playerLoc();
106    BOOST_CHECK_EQUAL(new_pos.x, init_pos.x + 3);
107    BOOST_CHECK_EQUAL(new_pos.y, init_pos.y);
108 }
109
110 // test for collision detection
111 BOOST_AUTO_TEST_CASE(test_collision) {
112     std::ifstream level("level2.lvl");
113     SB::Sokoban game;
114     level >> game;
115
116     // initial position should be (8, 5)
117     sf::Vector2u init_pos = game.playerLoc();
118
119     // move player up - boxes should stop them
120     game.movePlayer(SB::Direction::Up);
121     sf::Vector2u new_pos = game.playerLoc();
122     BOOST_CHECK_EQUAL(new_pos.x, init_pos.x);
123     BOOST_CHECK_EQUAL(new_pos.y, init_pos.y);
124
125     // move player left - wall should stop them
126     game.movePlayer(SB::Direction::Left);
127     new_pos = game.playerLoc();
128     BOOST_CHECK_EQUAL(new_pos.x, init_pos.x);
129     BOOST_CHECK_EQUAL(new_pos.y, init_pos.y);
130 }
131
132 // test for missing symbols

```

```

133 BOOST_AUTO_TEST_CASE(test_missing_symbol) {
134     std::ifstream level("swapoff.lvl");
135     SB::Sokoban game;
136     level >> game;
137
138     game.movePlayer(SB::Direction::Up);
139     game.movePlayer(SB::Direction::Up);
140     game.movePlayer(SB::Direction::Up);
141
142     BOOST_CHECK_EQUAL(game.playerLoc().y, 1);
143     BOOST_CHECK_EQUAL(game.playerLoc().x, 2);
144 }
145
146 // test victory conditions are satisfied when there are many boxes
147 BOOST_AUTO_TEST_CASE(test_victory_1) {
148     // load level5.lvl
149     std::ifstream level("level5.lvl");
150     SB::Sokoban game;
151     level >> game;
152     BOOST_CHECK_EQUAL(game.isWon(), false);
153
154     // path to complete level
155     game.movePlayer(SB::Direction::Right);
156     game.movePlayer(SB::Direction::Right);
157     game.movePlayer(SB::Direction::Right);
158     game.movePlayer(SB::Direction::Right);
159     game.movePlayer(SB::Direction::Up);
160     game.movePlayer(SB::Direction::Up);
161     game.movePlayer(SB::Direction::Up);
162     game.movePlayer(SB::Direction::Up);
163     game.movePlayer(SB::Direction::Left);
164     game.movePlayer(SB::Direction::Up);
165     game.movePlayer(SB::Direction::Right);
166
167     BOOST_CHECK(game.isWon());
168 }
169
170 // test victory conditions are satisfied when there are many targets
171 BOOST_AUTO_TEST_CASE(test_victory_2) {
172     // load level6.lvl
173     std::ifstream level("level6.lvl");
174     SB::Sokoban game;
175     level >> game;
176     BOOST_CHECK_EQUAL(game.isWon(), false);
177
178     // path to complete level
179     game.movePlayer(SB::Direction::Right);
180     game.movePlayer(SB::Direction::Up);
181     game.movePlayer(SB::Direction::Up);
182     game.movePlayer(SB::Direction::Up);
183     game.movePlayer(SB::Direction::Up);
184     game.movePlayer(SB::Direction::Right);
185     game.movePlayer(SB::Direction::Up);
186     game.movePlayer(SB::Direction::Left);
187     game.movePlayer(SB::Direction::Down);
188     game.movePlayer(SB::Direction::Down);
189     game.movePlayer(SB::Direction::Down);
190     game.movePlayer(SB::Direction::Down);
191     game.movePlayer(SB::Direction::Right);

```

```
192     game.movePlayer(SB::Direction::Right);
193     game.movePlayer(SB::Direction::Right);
194     game.movePlayer(SB::Direction::Up);
195     game.movePlayer(SB::Direction::Right);
196     game.movePlayer(SB::Direction::Down);
197
198     // game should now be won
199     BOOST_CHECK_EQUAL(game.isWon(), true);
200 }
```

6 PS5: DNA String Alignment

```
● jaydee@j4yd3ns14pt0p:~/comp4/project5$ ./EDistance < example10.txt
Edit distance: 7
A T 1
A A 0
C - 2
A A 0
G G 0
T G 1
T T 0
A - 2
C C 0
C A 1

Execution time is 0.000689 seconds
```

Figure 8: Output generated after running the code with sample .txt file

6.1 Assignment Overview

In this assignment, our objective was to write a program that computes the optimal alignment of two DNA strings in an input text file. This served as an introduction to computational biology, in which computers conduct research on biological systems, as well as a foundational programming concept known as *dynamic programming*, in which large computations are broken up into smaller, more manageable ones that are in turn solved and used to figure out the original objective. Pair programming was also encouraged for this assignment.

6.2 My Coding Approach

This was the last of three assignments that I ended up redoing to correct errors and incomplete sections. I used a two-dimensional matrix (represented as a vector of integers) to store the immediate values of the edit distance, with index i representing the first string and index j representing the second. The matrix is then filled using *recurrence relations*, expressions that analyze whether insertion, deleting, or substituting yields the least amount of cost. Once the matrix is filled, the optimal alignment is determined by adding characters from the two input strings, once again checking to see if they match, mismatch, or if a gap must be inserted.

The main methods of my EDistance class included `penalty()`, which returns 0 or depending on whether the characters in the string match; `min3()`, which returns the minimum of three integers; and `optDistance()`, which populates the two-dimensional matrix used to store the edit distance of the input strings. The `alignment()` function traces the matrix and modifies the strings to check if there is a match or mismatch, as well as if a gap needs to be inserted. However, since the matrix is assembled and parsed through from end to beginning, returning the accumulated string to the terminal wound up printing it backwards. Fortunately, this was a simple fix; I used `std::ostringstream` to reverse the vector of strings used to hold the alignment before returning it to the terminal.

6.3 What I Learned and Other Reflections

The intersection between healthcare and technology has always intrigued me, so the premise of this assignment alone was informative on what that intersection entails. As for the actual computations, the breakdown that I utilized in my implementation owing to recurrence relations and the general principles of dynamic programming were not as complicated as I thought it would be.

6.4 Source Code

Listing 27: Makefile

```

1 CC = g++
2 FLAGS = -Wall -Werror -std=c++17 -pedantic -g
3 LIBS = -lsfml-graphics -lsfml-window -lsfml-system -
        lboost_unit_test_framework
4
5 # .hpp files
6 DEPS = EDistance.hpp
7 # Compiled .o files
8 OBJ = EDistance.o
9 # Name of the program
10 PROGRAM = EDistance
11
12 .PHONY: all clean lint
13
14 all: $(PROGRAM) EDistance.a test
15
16 %.o: %.cpp $(DEPS)
17     $(CC) $(FLAGS) -c $< -o $@
18
19 main.o: main.cpp $(DEPS)
20     $(CC) $(FLAGS) -c main.cpp -o main.o
21
22 $(PROGRAM): main.o $(OBJ)
23     $(CC) $(FLAGS) -o $@ $^ $(LIBS)
24
25 test: test.o $(OBJ)
26     $(CC) $(FLAGS) -o $@ $^ $(LIBS)
27
28 EDistance.a: $(OBJ)
29     ar rcs $@ $^
30
31 clean:
32     rm -f *.o $(PROGRAM) EDistance.a test
33
34 lint:
35     cpplint *.cpp *.hpp

```

Listing 28: main.cpp

```

1 // Copyright 2025 by Jayden Danquah
2
3 #include <iostream>
4 #include <fstream>
5 #include <string>
6 #include <SFML/System.hpp>
7 #include "EDistance.hpp"
8
9 int main(int argc, char* argv[]) {
10     // clock to measure execution time
11     sf::Clock clock;
12
13     // read sequences from file into EDistance object
14     EDistance ed;
15     std::cin >> ed;
16
17     // get the alignment and edit distance
18     std::string alignment;
19     int distance = ed.optDistance();
20     alignment = ed.alignment();

```

```

21
22     // print the results
23     std::cout << "Edit distance: " << distance << std::endl;
24     std::cout << alignment << std::endl;
25
26     // get execution time
27     sf::Time t = clock.getElapsedTime();
28     std::cout << "Execution time is " << t.asSeconds() <<
29     " seconds" << std::endl;
30
31     return 0;
32 }

```

Listing 29: EDistance.cpp

```

1  // Copyright 2025 by Jayden Danquah
2
3  #include <iostream>
4  #include <string>
5  #include <fstream>
6  #include <vector>
7  #include <algorithm>
8  #include "EDistance.hpp"
9
10 // default constructor
11 EDistance::EDistance() : matrix(0, std::vector<int>(0, 0)) {}
12
13 // constructor that takes a filename
14 EDistance::EDistance(const std::string& filename) {
15     // open level file
16     std::ifstream file(filename);
17     if (!file) {
18         std::cerr << "ERROR - could not find " << filename << std::endl;
19         return;
20     }
21
22     // read first line into s1
23     std::getline(file, s1);
24
25     // read second line into s2
26     std::getline(file, s2);
27
28     // initialize matrix using string sizes
29     matrix.resize(s1.size() + 1, std::vector<int>(s2.size() + 1, 0));
30
31     // fill rows and columns with gap penalties
32     for (size_t i = 0; i <= s1.size(); ++i) {
33         matrix[i][0] = i * gap;
34     }
35     for (size_t j = 0; j <= s2.size(); ++j) {
36         matrix[0][j] = j * gap;
37     }
38 }
39
40 // two-string constructor
41 EDistance::EDistance(const std::string& s1, const std::string& s2)
42     : s1(s1), s2(s2) {
43     // initialize matrix using string sizes
44     matrix.resize(s1.size() + 1, std::vector<int>(s2.size() + 1, 0));
45

```

```

46 // fill rows and columns with gap penalties
47 for (size_t i = 0; i <= s1.size(); ++i) {
48     matrix[i][0] = i * gap;
49 }
50 for (size_t j = 0; j <= s2.size(); ++j) {
51     matrix[0][j] = j * gap;
52 }
53 }
54
55 // returns penalty for matched or mismatched characters
56 int EDistance::penalty(char a, char b) {
57     if (a == b) {
58         return 0;
59     } else {
60         return 1;
61     }
62 }
63
64 // return minimum of three values
65 int EDistance::min3(int a, int b, int c) {
66     return std::min({a, b, c});
67 }
68
69 // calculate edit distance between two strings
70 int EDistance::optDistance() {
71     for (size_t i = 1; i <= s1.length(); ++i) {
72         for (size_t j = 1; j <= s2.length(); ++j) {
73             int cost = penalty(s1[i - 1], s2[j - 1]);
74             matrix[i][j] = min3(
75                 matrix[i - 1][j] + gap,
76                 matrix[i][j - 1] + gap,
77                 matrix[i - 1][j - 1] + cost);
78         }
79     }
80     return matrix[s1.length()][s2.length()];
81 }
82
83 // find optimal alignment between two strings
84 std::string EDistance::alignment() const {
85     std::vector<std::string> align;
86     int i = s1.length();
87     int j = s2.length();
88
89     // trace through matrix
90     while (i > 0 || j > 0) {
91         // check if characters match or mismatch
92         if (i > 0 && j > 0 && matrix[i][j] ==
93             matrix[i - 1][j - 1] + penalty(s1[i - 1], s2[j - 1])) {
94             align.push_back(std::string(1, s1[i - 1]) + " " +
95                 s2[j - 1] + " " + std::to_string(penalty(s1[i - 1], s2[j - 1])))
96             ;
97             --i;
98             --j;
99         } // inserting a gap into s2
100         else if (i > 0 && (j == 0 ||
101             matrix[i][j] == matrix[i - 1][j] + gap)) {
102             align.push_back(std::string(1, s1[i - 1]) + " - "
103                 + std::to_string(gap));
104             --i;

```

```

104         // inserting a gap into s1
105     } else if (j > 0 && (i == 0 ||
106         matrix[i][j] == matrix[i][j - 1] + gap)) {
107         align.push_back("- " + std::string(1, s2[j - 1]) + " "
108         + std::to_string(gap));
109         --j;
110     }
111 }
112
113 // return accumulated string
114 std::ostringstream oss;
115 for (auto it = align.rbegin(); it != align.rend(); ++it) {
116     oss << *it << "\n";
117 }
118 return oss.str();
119 }
120
121 // print the matrix
122 void EDistance::print_matrix() const {
123     for (const auto& row : matrix) {
124         for (const auto& val : row) {
125             std::cout << std::setw(3) << val << " ";
126         }
127         std::cout << std::endl;
128     }
129 }
130
131 // overloaded operators
132 std::istream& operator>>(std::istream& is, EDistance& ed) {
133     std::string a, b;
134     is >> a >> b;
135     ed = EDistance(a, b);
136     return is;
137 }
138
139 std::ostream& operator<<(std::ostream& os, const EDistance& ed) {
140     os << ed.alignment();
141     return os;
142 }

```

Listing 30: EDistance.hpp

```

1 // Copyright 2025 by Jayden Danquah
2
3 #pragma once
4
5 #include <iostream>
6 #include <algorithm>
7 #include <string>
8 #include <vector>
9 #include <iomanip>
10
11 class EDistance {
12 public:
13     // constructors (default, filename, two-string)
14     EDistance();
15     explicit EDistance(const std::string& filename);
16     EDistance(const std::string& s1, const std::string& s2);
17
18     static int penalty(char a, char b);

```

```

19     static int min3(int a, int b, int c);
20
21     int optDistance();
22     std::string alignment() const;
23
24     std::vector<std::vector<int>> get_matrix() const { return matrix; }
25     void print_matrix() const;
26
27     // overloaded operators
28     friend std::istream& operator>>(std::istream& is, EDistance& ed);
29     friend std::ostream& operator<<(std::ostream& os, const EDistance& ed);
30
31 private:
32     std::string s1;
33     std::string s2;
34     int match = 0;
35     int mismatch = 1;
36     int gap = 2;
37     std::vector<std::vector<int>> matrix;
38 };

```

Listing 31: test.cpp

```

1  // Copyright 2025 by Jayden Danquah
2
3  #include <iostream>
4  #include <string>
5  #include <vector>
6  #include <sstream>
7  #include <algorithm>
8  #include "EDistance.hpp"
9
10 #define BOOST_TEST_DYN_LINK
11 #define BOOST_TEST_MODULE Main
12 #include <boost/test/unit_test.hpp>
13
14 // test alignment of two strings
15 BOOST_AUTO_TEST_CASE(test_alignment_1) {
16     std::string s1 = "kitten";
17     std::string s2 = "sitting";
18
19     EDistance ed(s1, s2);
20     int distance = ed.optDistance();
21     BOOST_CHECK_EQUAL(distance, 4);
22
23     std::string result = ed.alignment();
24
25     std::string top = "k s 1";
26     std::string bottom = "i i 0";
27
28     std::istringstream iss(result);
29     std::string top_line, bottom_line;
30     std::getline(iss, top_line);
31     std::getline(iss, bottom_line);
32
33     BOOST_CHECK_EQUAL(top_line, top);
34     BOOST_CHECK_EQUAL(bottom_line, bottom);
35
36     BOOST_CHECK_EQUAL(top_line.length(), bottom_line.length());
37

```

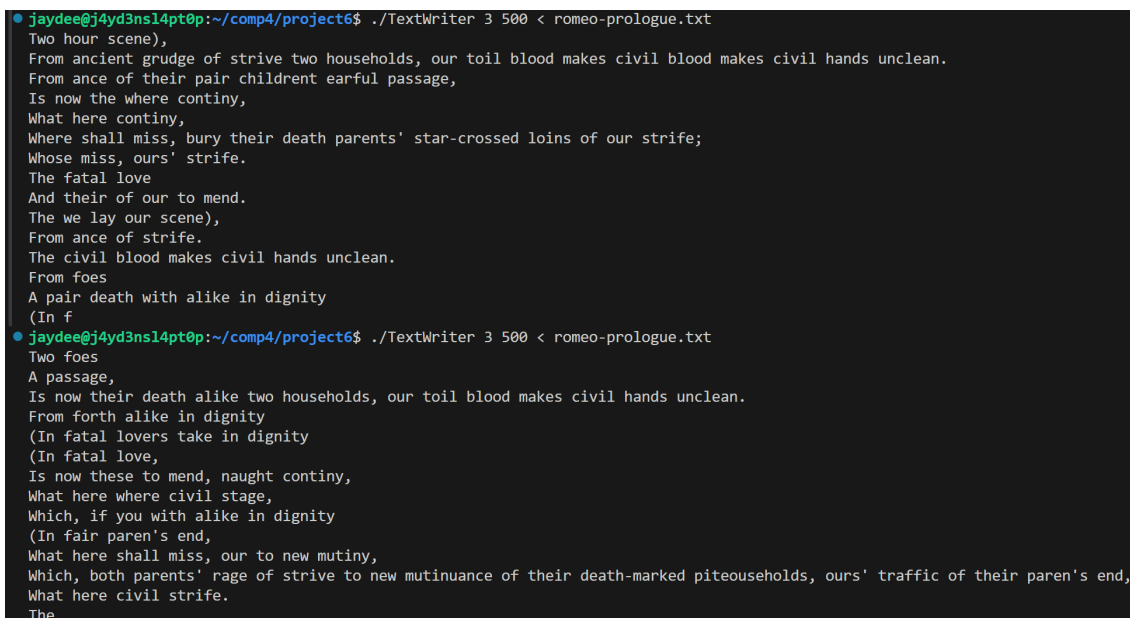
```

38     BOOST_CHECK_EQUAL(top_line.back(), '1');
39     BOOST_CHECK_EQUAL(bottom_line.back(), '0');
40 }
41
42 // test alignment of two strings in reverse order
43 BOOST_AUTO_TEST_CASE(test_alignment_2) {
44     std::string s1 = "sitting";
45     std::string s2 = "kitten";
46
47     EDistance ed(s1, s2);
48     int distance = ed.optDistance();
49     BOOST_CHECK_EQUAL(distance, 4);
50
51     std::string result = ed.alignment();
52
53     std::string top = "s k 1";
54     std::string bottom = "i i 0";
55
56     std::istringstream iss(result);
57     std::string top_line, bottom_line;
58     std::getline(iss, top_line);
59     std::getline(iss, bottom_line);
60
61     BOOST_CHECK_EQUAL(top_line, top);
62     BOOST_CHECK_EQUAL(bottom_line, bottom);
63
64     BOOST_CHECK_EQUAL(top_line.length(), bottom_line.length());
65
66     BOOST_CHECK_EQUAL(top_line.back(), '1');
67     BOOST_CHECK_EQUAL(bottom_line.back(), '0');
68 }
69
70 // tests min3 function
71 BOOST_AUTO_TEST_CASE(test_min3) {
72     // middle is the minimum
73     BOOST_CHECK_EQUAL(EDistance::min3(5, 3, 8), 3);
74
75     // first is the minimum
76     BOOST_CHECK_EQUAL(EDistance::min3(1, 2, 3), 1);
77
78     // last is the minimum
79     BOOST_CHECK_EQUAL(EDistance::min3(9, 8, 7), 7);
80
81     // all are equal
82     BOOST_CHECK_EQUAL(EDistance::min3(4, 4, 4), 4);
83
84     // two are equal
85     BOOST_CHECK_EQUAL(EDistance::min3(1, 1, 4), 1);
86     BOOST_CHECK_EQUAL(EDistance::min3(5, 2, 2), 2);
87     BOOST_CHECK_EQUAL(EDistance::min3(3, 6, 3), 3);
88 }
89
90 // test the cost function
91 BOOST_AUTO_TEST_CASE(test_cost) {
92     // no cost
93     EDistance ed1("a", "a");
94     BOOST_CHECK_EQUAL(ed1.optDistance(), 0);
95
96     // cost 1

```

```
97     EDistance ed2("a", "b");
98     BOOST_CHECK_EQUAL(ed2.optDistance(), 1);
99
100     // gap cost 2 (insertion and deletion)
101     EDistance ed3("a", "ab");
102     BOOST_CHECK_EQUAL(ed3.optDistance(), 2);
103     EDistance ed4("ab", "a");
104     BOOST_CHECK_EQUAL(ed4.optDistance(), 2);
105
106     // mismatch and gap
107     EDistance ed5("ab", "cb");
108     BOOST_CHECK_EQUAL(ed5.optDistance(), 1);
109 }
```

7 PS6: RandWriter



```
• jaydee@j4yd3ns14pt0p:~/comp4/project6$ ./TextWriter 3 500 < romeo-prologue.txt
Two hour scene),
From ancient grudge of strive two households, our toil blood makes civil blood makes civil hands unclean.
From ance of their pair childrent earful passage,
Is now the where continy,
What here continy,
Where shall miss, bury their death parents' star-crossed loins of our strife;
Whose miss, ours' strife.
The fatal love
And their of our to mend.
The we lay our scene),
From ance of strife.
The civil blood makes civil hands unclean.
From foes
A pair death with alike in dignity
(In f
• jaydee@j4yd3ns14pt0p:~/comp4/project6$ ./TextWriter 3 500 < romeo-prologue.txt
Two foes
A passage,
Is now their death alike two households, our toil blood makes civil hands unclean.
From forth alike in dignity
(In fatal lovers take in dignity
(In fatal love,
Is now these to mend, naught continy,
What here where civil stage,
Which, if you with alike in dignity
(In fair paren's end,
What here shall miss, our to new mutiny,
Which, both parents' rage of strive to new mutinuanace of their death-marked piteouseholds, ours' traffic of their paren's end,
What here civil strife.
The
```

Figure 9: Examples of nonsense text generated after running the code

7.1 Assignment Overview

In this assignment, our objective was to analyze input text for transitions between k-grams (a fixed number of characters) and the following the letter, then produce a probabilistic string.

7.2 My Coding Approach

The constructor analyzes the input text by extracting every possible k-gram and records the characters that follow each k-gram along with their frequencies. This information is then organized into a frequency map and a general k-gram map, allowing the `kRand()` function to pick the next likely character following a given k-gram. The `generate()` function utilizes a k-gram as a seed, then calls `kRand()` to determine the next character, appending it to an output string; this repeats until the output string reaches the desired length, `L`. Within `generate()`, a local lambda function named `append_char()` captures the logic for obtaining the next character and updating the current k-gram, allowing the main loop to concentrate solely on verifying the output length.

Some issues I ran into while developing this code included initially writing `freq()` to return the total occurrences of a k-gram rather than the actual frequency of the character that follows it. Additionally, I used `srand()` from the `ctime` library to generate unique outputs each time the program ran, but my implementation of `kRand()` caused the random generation to reset with each call. To solve this, I changed my logic so that random generation was instead handled by a class variable, with the help of `std::default_random_engine` and `std::uniform_int_distribution`. Lastly, my first pass at the circular logic used to construct the k-gram unintentionally omitted some k-grams; in other words, it was essentially "half-circular".

7.3 What I Learned and Other Reflections

`srand()` was the go-to function I used in previous C/C++ assignments to generate randomized input, but getting to know about `std::default_random_engine` and `std::uniform_int_distribution` was of substantial help since `srand()` proved that it is not always reliable when handling large or generally complex programming problems such as working with k-grams.

7.4 Source Code

Listing 32: Makefile

```

1 CC = g++
2 FLAGS = -Wall -Werror -std=c++17 -pedantic -g
3 LIBS = -lboost_unit_test_framework
4
5 # .hpp files
6 DEPS = RandWriter.hpp
7
8 # Compiled .o files
9 OBJ = RandWriter.o TextWriter.o
10
11 # Names
12 PROGRAM = TextWriter
13 TEST = test
14 LIBRARY = TextWriter.a
15
16 .PHONY: all clean lint
17
18 all: $(PROGRAM) $(LIBRARY) $(TEST)
19
20 %.o: %.cpp $(DEPS)
21     $(CC) $(FLAGS) -c $< -o $@
22
23 # Build program
24 $(PROGRAM): RandWriter.o TextWriter.o
25     $(CC) $(FLAGS) -o $@ RandWriter.o TextWriter.o $(LIBS)
26
27 # Build test program
28 $(TEST): RandWriter.o test.o
29     $(CC) $(FLAGS) -o $@ RandWriter.o test.o $(LIBS)
30
31 # Build static library
32 $(LIBRARY): RandWriter.o
33     ar rcs $@ RandWriter.o
34
35 clean:
36     rm -f *.o $(PROGRAM) $(LIBRARY) $(TEST)
37
38 lint:
39     cpplint *.cpp *.hpp

```

Listing 33: TextWriter.cpp

```

1 // Copyright 2025 by Jayden Danquah
2
3 #include <iostream>
4 #include <sstream>
5 #include <string>
6 #include "RandWriter.hpp"
7
8 int main(int argc, char* argv[]) {
9     // get command-line arguments
10    if (argc != 3) {
11        std::cerr << "Usage: ./TextWriter <order k> <length L> < input.txt"
12        << std::endl;
13        return 1;
14    }
15    size_t k = std::stoi(argv[1]), L = std::stoi(argv[2]);
16
17    // read input into string

```

```

18     std::stringstream buffer;
19     buffer << std::cin.rdbuf();
20     std::string input = buffer.str();
21
22     if (input.length() < k) {
23         std::cerr << "ERROR - input must be as long as k." << std::endl;
24         return 1;
25     }
26     if (L < k) {
27         std::cerr << "ERROR - length L must be at least k." << std::endl;
28         return 1;
29     }
30
31     // create RandWriter object (Markov model)
32     RandWriter model(input, k);
33
34     // get kgram from first k characters
35     std::string seed = input.substr(0, k);
36
37     // generate string
38     std::string output = model.generate(seed, L);
39     std::cout << output << std::endl;
40
41     return 0;
42 }

```

Listing 34: RandWriter.cpp

```

1  // Copyright 2025 by Jayden Danquah
2
3  #include <random>
4  #include <string>
5  #include <map>
6  #include <stdexcept>
7  #include <ctime>
8  #include "RandWriter.hpp"
9
10 // constructor
11 RandWriter::RandWriter(const std::string& str, size_t k) {
12     // check that string length is valid
13     if (str.length() < k) {
14         throw std::invalid_argument("ERROR - string length must be at least
15 k");
16     }
17
18     size_t n = str.length();
19     this->k = k;
20     this->text = str;
21     generator.seed(time(nullptr));
22
23     // initialize frequency and kgram maps
24     for (size_t i = 0; i < n; i++) {
25         // create kgram by wrapping around string
26         std::string kgram;
27         for (size_t j = 0; j < k; j++) {
28             kgram += str[(i + j) % n];
29         }
30
31         // get next character
32         char next_char = str[(i + k) % n];

```

```

32
33     // update frequency map
34     frequency_map[kgram]++;
35
36     // update kgram map
37     kgram_map[kgram][std::string(1, next_char)]++;
38 }
39 }
40
41 // number of occurrences of kgram in text
42 int RandWriter::freq(const std::string& kgram) const {
43     if (kgram.length() != k) {
44         throw std::invalid_argument("ERROR - kgram must be of length k");
45     }
46     auto it = frequency_map.find(kgram);
47     return (it != frequency_map.end()) ? it->second : 0;
48 }
49
50 // number of times c follows kgram
51 int RandWriter::freq(const std::string& kgram, char c) const {
52     if (kgram.length() != k) {
53         throw std::invalid_argument("ERROR - kgram must be of length k");
54     }
55
56     // look for kgram in kgram_map
57     auto it = kgram_map.find(kgram);
58     if (it != kgram_map.end()) {
59         // look for character c following kgram
60         auto sub_it = it->second.find(std::string(1, c));
61         return (sub_it != it->second.end()) ? sub_it->second : 0;
62     }
63     return 0;
64 }
65
66 // random character following kgram
67 char RandWriter::kRand(const std::string& kgram) {
68     if (kgram.length() != k) {
69         throw std::invalid_argument("ERROR - kgram must be of length k");
70     }
71
72     // look for kgram in map
73     auto it = kgram_map.find(kgram);
74     if (it == kgram_map.end()) {
75         throw std::invalid_argument("ERROR - kgram does not exist in model");
76     }
77
78     // total frequency of characters following kgram
79     int total = 0;
80     for (const auto& p : it->second) {
81         total += p.second;
82     }
83
84     // get random number using uniform distribution
85     std::uniform_int_distribution<int> distribution(0, total - 1);
86     int r = distribution(generator);
87
88     // find character corresponding to random number
89     int cumulative = 0;

```

```

90     for (const auto& p : it->second) {
91         cumulative += p.second;
92         if (r < cumulative) {
93             return p.first[0];
94         }
95     }
96
97     return it->second.begin()->first[0];
98 }
99
100 // generate string of length L
101 std::string RandWriter::generate(const std::string& kgram, size_t l) {
102     if (kgram.length() != k) {
103         throw std::invalid_argument("ERROR - kgram must be of length k");
104     }
105     if (l < k) {
106         throw std::invalid_argument("ERROR - length L must be at least k");
107     }
108
109     std::string output = kgram;
110     std::string current = kgram;
111
112     // lambda function to append random character
113     auto append_char = [&]() {
114         char next_char = kRand(current);
115         output += next_char;
116         current = output.substr(output.size() - k, k);
117     };
118
119     // append characters until output.length() = L
120     while (output.length() < l) {
121         append_char();
122     }
123
124     return output;
125 }

```

Listing 35: RandWriter.hpp

```

1 // Copyright 2025 by Jayden Danquah
2
3 #pragma once
4
5 #include <string>
6 #include <map>
7 #include <random>
8
9 class RandWriter {
10 public:
11     // Create a Markov model of order k from given text
12     // Assume that text has length at least k.
13     RandWriter(const std::string& str, size_t k);
14
15     // Order k of Markov model
16     size_t orderK() const { return k; }
17
18     // Number of occurrences of kgram in text
19     // Throw an exception if kgram is not length k
20     int freq(const std::string& kgram) const;
21

```

```

22 // Number of times that character c follows kgram
23 // if order=0, return num of times that char c appears
24 // (throw an exception if kgram is not of length k)
25 int freq(const std::string& kgram, char c) const;
26
27 // Random character following given kgram
28 // (throw an exception if kgram is not of length k)
29 // (throw an exception if no such kgram)
30 char kRand(const std::string& kgram);
31
32 // Generate a string of length L characters by simulating a trajectory
33 // through the corresponding Markov chain. The first k characters of
34 // the newly generated string should be the argument kgram.
35 // Throw an exception if kgram is not of length k.
36 // Assume that L is at least k
37 std::string generate(const std::string& kgram, size_t l);
38
39 private:
40     std::map<std::string, std::map<std::string, int>> kgram_map;
41     std::map<std::string, int> frequency_map;
42     size_t k;
43     std::string text;
44     std::default_random_engine generator;
45 };

```

Listing 36: test.cpp

```

1 // Copyright 2025 by Jayden Danquah
2
3 #include <string>
4 #include <fstream>
5 #include <sstream>
6 #include "RandWriter.hpp"
7
8 #define BOOST_TEST_DYN_LINK
9 #define BOOST_TEST_MODULE Main
10 #include <boost/test/unit_test.hpp>
11
12 // test that constructor works
13 BOOST_AUTO_TEST_CASE(test_constructor) {
14     std::string input = "hellohello";
15     size_t k = 2;
16     RandWriter model(input, k);
17     BOOST_CHECK_EQUAL(model.orderK(), k);
18 }
19
20 // test that orderK() works
21 BOOST_AUTO_TEST_CASE(test_orderK) {
22     RandWriter model("abcabcabc", 3);
23     BOOST_CHECK_EQUAL(model.orderK(), 3);
24 }
25
26 // test that 1st freq() overload works
27 BOOST_AUTO_TEST_CASE(test_freq_1) {
28     RandWriter model("ababab", 2);
29     BOOST_CHECK_EQUAL(model.freq("ab"), 3);
30     BOOST_CHECK_EQUAL(model.freq("ba"), 3);
31 }
32
33 // test that generate() works as intended

```

```

34 BOOST_AUTO_TEST_CASE(test_generate) {
35     RandWriter model("hellohello", 2);
36     std::string seed = "he";
37     size_t L = 10;
38
39     std::string result = model.generate(seed, L);
40
41     // check length is correct
42     BOOST_CHECK_EQUAL(result.length(), L);
43
44     // check that first k characters are seed
45     BOOST_CHECK_EQUAL(result.substr(0, seed.length()), seed);
46
47     // check for substrings of length k in result
48     for (size_t i = 0; i <= result.size() - model.orderK(); i++) {
49         std::string kgram = result.substr(i, model.orderK());
50         BOOST_CHECK(model.freq(kgram) > 0);
51     }
52 }
53
54 // test for properly throwing invalid arguments
55 BOOST_AUTO_TEST_CASE(test_invalid) {
56     // invalid constructor
57     BOOST_REQUIRE_THROW(RandWriter model("hi", 5), std::invalid_argument);
58
59     // invalid kgram length
60     RandWriter model2("hellohello", 3);
61     BOOST_REQUIRE_THROW(model2.generate("he", 10), std::invalid_argument);
62 }

```

8 PS7: Kronos InTouch Log Parsing

```
1 Device Boot Report
2
3 InTouch log file: device1_intouch.log
4 Lines Scanned: 443838
5
6 Device boot count: initiated = 6, completed: 6
7
8
9 === Device boot ===
10 435369(device1_intouch.log): 2014-03-25 19:11:59 Boot Start
11 435759(device1_intouch.log): 2014-03-25 19:15:02 Boot Completed
12 | Boot Time: 183369ms
13
14 === Device boot ===
15 436500(device1_intouch.log): 2014-03-25 19:29:59 Boot Start
16 436859(device1_intouch.log): 2014-03-25 19:32:44 Boot Completed
17 | Boot Time: 165036ms
18
19 === Device boot ===
20 440719(device1_intouch.log): 2014-03-25 22:01:46 Boot Start
21 440791(device1_intouch.log): 2014-03-25 22:04:27 Boot Completed
22 | Boot Time: 161514ms
23
24 === Device boot ===
25 440866(device1_intouch.log): 2014-03-26 12:47:42 Boot Start
26 441216(device1_intouch.log): 2014-03-26 12:50:29 Boot Completed
27 | Boot Time: 167824ms
28
29 === Device boot ===
30 442094(device1_intouch.log): 2014-03-26 20:41:34 Boot Start
31 442432(device1_intouch.log): 2014-03-26 20:44:13 Boot Completed
32 | Boot Time: 159235ms
33
34 === Device boot ===
35 443073(device1_intouch.log): 2014-03-27 14:09:01 Boot Start
36 443411(device1_intouch.log): 2014-03-27 14:11:42 Boot Completed
37 | Boot Time: 161500ms
38
39
```

Figure 10: Report file generated after running the code

8.1 Assignment Overview

For our final project of the semester, we were given the option to code our program in either C++ or Python using the regular expressions library (or regex for short). Our objective was to analyze the log files of the Kronos InTouch, a Linux-based touch screen time clock, and report back the startup times and the status of the boot in a new report file.

8.2 My Coding Approach

Before writing my implementation, I analyzed the logs and reports provided to us in the .zip files to understand how I was going to arrange the formatting for my report. The InTouch boot sequence is indicated by the line “(log.c.166) server started”, while its end is marked by “oejs.AbstractConnector:StartedSelectChannelConnector@0.0.0.0:9080”. From there, I created regex patterns to match the boot start and end indicators, and then created two additional patterns to capture the actual times of the boot start and end. Once these patterns were established, I read a sample log into my program and then parsed through it

to find the desired occurrences using the regex patterns. Each time an instance was found, it was formatted to resemble the instances in the report files, and then those were written to a brand new output file. The output file also contains a header summary for extra credit, displaying the log file that was just read, the number of lines that were scanned from the file, and finally a counter to check how many boot sequences were initiated and completed during the parsing. All of those values were tracked and stored in variables that I set before my main loop.

8.3 What I Learned and Other Reflections

Python is a programming language that I have moderate experience with from middle and high school, but this project was the first time I directly worked with its libraries in the way I have worked with C/C++ and its libraries in the past. The simplified syntax of Python made it easier to construct the general outline of my implementation, and setting up the regex patterns was easy as well.

8.4 Source Code

Listing 37: Makefile

```
1 all: ps7
2
3 ps7: ps7.py
4     echo '#!/bin/bash' > ps7
5     echo 'python3 ps7.py "$$@"' >> ps7
6     chmod +x ps7
7
8 lint:
9     flake8 ps7.py
10
11 clean:
12     rm -f ps7
```

Listing 38: ps7.py

```
1 # Copyright 2025 by Jayden Danquah
2
3 import sys
4 import re
5 from datetime import datetime
6
7 # get arguments from command line
8 if len(sys.argv) != 2:
9     print("Usage: ./ps7 <filename>")
10    sys.exit(1)
11 filename = sys.argv[1]
12 output_file = filename + ".rpt"
13
14 # regex patterns
15 START = re.compile(r'\(log\.c\.166\) server started')
16 END = re.compile(r'oejs\.AbstractConnector:Started SelectChannelConnector@')
17 TIME_START = re.compile(r'\(\\d{4}-\\d{2}-\\d{2} \\d{2}:\\d{2}:\\d{2}):')
18 TIME_END = re.compile(r'\(\\d{4}-\\d{2}-\\d{2} \\d{2}:\\d{2}:\\d{2})\\.\\d{3})')
19
20 # read lines from file
21 with open(filename, 'r') as file:
22     lines = file.readlines()
23 line_count = len(lines)
24 print("InTouch log file: ", filename)
25 print("Lines scanned: ", line_count)
26
```

```

27 # boot process variables
28 boot_init = 0
29 boot_done = 0
30 output = []
31 boot_start = False
32
33 # find start and end of lines
34 for i, line in enumerate(lines):
35     # check for start of log
36     if START.search(line):
37         if boot_start:
38             output.append(("**** Incomplete boot ****",))
39         match = TIME_START.search(line)
40         if match:
41             time_start_str = match.group(1)
42             output.append(("=== Device boot ===", i + 1, time_start_str))
43             boot_start = True
44             boot_init += 1
45
46     # check for end of log
47     if boot_start and END.search(line):
48         match = TIME_END.search(line)
49         if match:
50             time_end_str = match.group(1)
51             # find last instance of start
52             for j in reversed(output):
53                 if isinstance(j, tuple) and j[0] == "=== Device boot ===":
54                     time_start_str = j[2]
55                     break
56             # calculate boot time
57             time_start = datetime.strptime(time_start_str, "%Y-%m-%d %H:%M:%
S")
58             time_end = datetime.strptime(time_end_str, "%Y-%m-%d %H:%M:%S.%f
")
59             duration = int((time_end - time_start).total_seconds() * 1000)
60
61             output.append((i + 1, time_end.strftime("%Y-%m-%d %H:%M:%S"), "
Boot Completed"))
62             output.append((f"\tBoot Time: {duration}ms\n",))
63             boot_done += 1
64             boot_start = False
65
66 # write to output file
67 with open(output_file, 'w') as file:
68     # summary header
69     file.write(f"Device Boot Report\n\n")
70     file.write(f"InTouch log file: {filename}\n")
71     file.write(f"Lines Scanned: {line_count}\n\n")
72     file.write(f"Device boot count: initiated = {boot_init}, completed: {
boot_done}\n\n\n")
73
74     for line in output:
75         if line[0] == "=== Device boot ===":
76             file.write(f"{line[0]}\n")
77             file.write(f"{line[1]}({filename}): {line[2]} Boot Start\n")
78         elif line[0] == "**** Incomplete boot ****":
79             file.write(f"{line[0]}\n\n")
80         elif len(line) == 3:
81             file.write(f"{line[0]}({filename}): {line[1]} {line[2]}\n")

```

```
82     elif len(line) == 1:  
83         file.write(f"{line[0]}\n")
```